



UNIVERSITÀ  
POLITECNICA  
DELLE MARCHE

**Università Politecnica delle Marche**

Facoltà di Ingegneria Informatica e dell'Automazione

---

# Classification, Clustering & Timeseries

Analisi dei crimini nella città di Baltimora

**Autori:**

Matteo Sonaglioni

Enzo Cingoli

Gabriel Vladut Popa

Anno Accademico 2025–2026

*Relazione redatta per il corso di Data Science*

# Indice

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Crimes in Baltimora</b>                                        | <b>3</b>  |
| 1.1      | Crimes in Baltimora Analysis . . . . .                            | 3         |
| 1.2      | Data Preprocessing ed ETL (Extract, Transform, Load) . . . . .    | 3         |
| 1.2.1    | Fase di pulizia e allineamento iniziale . . . . .                 | 3         |
| <b>2</b> | <b>Classification</b>                                             | <b>6</b>  |
| 2.1      | Preparazione dati . . . . .                                       | 6         |
| 2.2      | Training . . . . .                                                | 9         |
| 2.3      | Risultati della Classificazione . . . . .                         | 10        |
| 2.4      | Classificazione con SMOTEENN . . . . .                            | 13        |
| 2.5      | Risultati della Classificazione con SMOTEENN . . . . .            | 15        |
| 2.6      | Conclusioni e osservazioni . . . . .                              | 18        |
| <b>3</b> | <b>Clustering</b>                                                 | <b>19</b> |
| 3.1      | Preparazione dati . . . . .                                       | 19        |
| 3.2      | Clustering dei distretti con K-Means . . . . .                    | 21        |
| 3.3      | Analisi visiva dei cluster tramite PCA . . . . .                  | 22        |
| 3.4      | Interpretazione dei centroidi . . . . .                           | 25        |
| 3.5      | Analisi quantitativa tramite Lift . . . . .                       | 28        |
| 3.6      | Analisi gerarchica dei distretti . . . . .                        | 31        |
| 3.7      | Visualizzazione geografica dei cluster . . . . .                  | 33        |
| 3.8      | Clustering con DBSCAN per l'identificazione di Hotspots . . . . . | 34        |
| 3.9      | Conclusioni generali . . . . .                                    | 37        |
| <b>4</b> | <b>Timeseries</b>                                                 | <b>39</b> |
| 4.1      | Preparazione dati . . . . .                                       | 39        |
| 4.2      | Analisi esplorativa del dataset . . . . .                         | 40        |
| 4.3      | Analisi dei correlogrammi e delle componenti . . . . .            | 47        |
| 4.4      | Addestramento del modello . . . . .                               | 53        |
| 4.5      | Miglioramento del modello SARIMAX . . . . .                       | 55        |
| 4.6      | Conclusioni ed osservazioni . . . . .                             | 62        |

# 1 Crimes in Baltimora

## 1.1 Crimes in Baltimora Analysis

Le presenti analisi si fondano sull'utilizzo del dataset "City of Baltimore Crime Data". Tale raccolta di dati offre una granularità temporale e spaziale significativa, includendo eventi criminali registrati nella città di **Baltimora (Maryland, USA)** in un intervallo esteso compreso tra il **2010 e il 2017**. Ogni record documenta in dettaglio eventi specifici, fornendo variabili essenziali quali la **data e l'ora** dell'evento, la **tipologia di reato**, la **localizzazione geografica** e l'identificazione del **distretto di polizia** di competenza. La ricchezza e la multidimensionalità di queste informazioni lo rendono un *corpus* ideale per l'applicazione di metodologie di **Analisi delle Serie Temporal**i per la previsione dell'andamento evolutivo dei crimini aggregati, il **Clustering** per l'identificazione di profili criminali dei vari distretti e la **Classification** per la categorizzazione di variabili correlate all'evento.

## 1.2 Data Preprocessing ed ETL (Extract, Transform, Load)

### 1.2.1 Fase di pulizia e allineamento iniziale

La fase di *Extract* e la pulizia iniziale sono state uniformi per tutti gli approcci di modellazione. Il processo ha avuto inizio con il caricamento del dataset in formato ".csv".

Il dataset analizzato contiene **285.807** osservazioni e **12** colonne.

La seguente tabella (Tabella 1) riassume la struttura e la descrizione delle variabili presenti nel dataset:

Tabella 1: Struttura e descrizione delle colonne del dataset Crime in Baltimora.

| Colonna                | Tipo    | Descrizione                                                                                            |
|------------------------|---------|--------------------------------------------------------------------------------------------------------|
| <b>CrimeDate</b>       | object  | Data in cui il crimine è stato registrato.                                                             |
| <b>CrimeTime</b>       | object  | Ora dell'evento.                                                                                       |
| <b>CrimeCode</b>       | object  | Codice identificativo del tipo di reato.                                                               |
| <b>Location</b>        | object  | Indirizzo o descrizione del luogo. Alcuni valori mancanti ( $\approx 0.6\%$ ).                         |
| <b>Description</b>     | object  | Descrizione testuale del tipo di crimine (es. "ROBBERY").                                              |
| <b>Inside/Outside</b>  | object  | Indica se il crimine è avvenuto all'interno o all'esterno. Alcuni valori mancanti ( $\approx 1.5\%$ ). |
| <b>Weapon</b>          | object  | Tipologia di arma utilizzata. Alta percentuale di valori mancanti ( $\sim 66\%$ ).                     |
| <b>Post</b>            | float64 | Numero del distretto di polizia (post).                                                                |
| <b>District</b>        | object  | Nome del distretto di polizia (es. "SOUTHERN"). Pochi valori mancanti.                                 |
| <b>Neighborhood</b>    | object  | Nome del quartiere di Baltimora. Alcuni valori mancanti ( $\approx 0.6\%$ ).                           |
| <b>Location 1</b>      | object  | Coordinate geografiche (latitudine, longitudine) in formato stringa.                                   |
| <b>Total Incidents</b> | int64   | Numero totale di incidenti registrati (spesso pari a 1).                                               |

Il dataset si presenta **abbastanza completo**, con una percentuale di valori nulli limitata in quasi tutte le colonne ad eccezione di **Weapon**, il che impone particolare attenzione durante la fase di pulizia e nella scelta del **target** per la classificazione. I dati sono **prevalentemente categoriali e temporali**, rendendoli particolarmente adatti per l'applicazione di **modelli di forecasting** sul numero di incidenti nel tempo.

### 1. Gestione dei Valori Mancanti

- Le colonne **Location**, **Inside/Outside**, **Weapon**, **District** e **Neighborhood** presentano una certa quantità di valori nulli.

- Si è scelto di sostituire tali valori con una categoria "Unknown" per preservare il numero di osservazioni e non introdurre distorsioni, ad eccezione dei casi in cui la colonna era usata come target (e.g., Weapon nella classificazione).

## 2. Creazione della Colonna Temporale Unificata

- Le colonne CrimeDate e CrimeTime sono state unite in un'unica colonna Datetime con la seguente operazione:

```
1 crime_data['Datetime'] = pd.to_datetime(
2     crime_data['CrimeDate'] + ' ' + crime_data['CrimeTime'],
3     errors='coerce')
```

- Questa nuova variabile temporale è cruciale per:
  - ordinare gli eventi nel tempo (`sort_values('Datetime')`);
  - effettuare analisi stagionali o giornaliere;
  - eseguire *resampling* (su base giornaliera o settimanale) per l'analisi delle serie storiche.

## 3. Analisi Temporale e Aggregazione

- Per studiare l'evoluzione dei crimini nel tempo, il dataset è stato aggregato per periodo (settimanale) per generare la serie temporale:

```
1 crime_time_series = (
2     crime_data.set_index('Datetime')
3     .resample('W')['Total Incidents']
4     .sum())
```

- Questa operazione consente di generare una serie temporale settimanale, fondamentale per il **forecasting** del numero totale di crimini.

## 4. Verifica e Coerenza dei Dati Temporali

- Infine, i dati sono stati ordinati e l'indice è stato resettato per assicurare la coerenza temporale in preparazione per l'analisi sequenziale:

```
1 crime_data = (
2     crime_data.sort_values('Datetime')
3     .reset_index(drop=True))
```

Dopo questa fase generale, il flusso di *Transformation e Load* si è differenziato per soddisfare i requisiti algoritmici specifici di ciascuna analisi.

## 2 Classification

L'obiettivo dell'analisi di **Classificazione** consiste nel prevedere la tipologia di arma impiegata durante ciascun evento criminale, rappresentata dalla variabile **Weapon**. Il modello sviluppato ha il compito di assegnare ogni osservazione a una delle seguenti classi: *HANDS* (classe 1), *KNIFE* (classe 2), *FIREARM* (classe 0) o *OTHER* (classe 3).

Per la realizzazione del processo di analisi sono state impiegate le principali librerie del linguaggio **Python**, tra cui **pandas**, **matplotlib**, **seaborn**, **statsmodels**, **numpy**, **imblearn** e **scikit-learn**. Tali strumenti hanno consentito di eseguire una **pulizia approfondita dei dati**, un'accurata **analisi esplorativa** e la successiva **modellazione predittiva**, finalizzata alla costruzione e valutazione di modelli di classificazione.

### 2.1 Preparazione dati

Il passaggio critico è il filtraggio iniziale del dataset, necessario perché la classe **Unknown** (che è il valore dominante, con circa il 66% di valori mancanti) è stata **rimossa dal training set** per non falsare l'addestramento, riducendo significativamente la dimensione del dataset finale di classificazione.

Data la natura del task e la varietà delle categorie è stato adottato il **Label Encoding** per convertire le variabili testuali in formato numerico discreto.

```
1 categorical_cols = ['CrimeCode', 'Location', 'Description',  
2                   'Weapon', 'District', 'Neighborhood', 'DayOfWeek']  
3  
4 encoders = {}  
5  
6 for col in categorical_cols:  
7     le = LabelEncoder()  
8     crime_data[f'{col}_encoded'] = le.fit_transform(crime_data[col])  
9     encoders[col] = le
```

Inoltre, è stata eseguita una **selezione preliminare delle feature** (non strettamente necessaria ma utile per l'interpretabilità) eliminando quelle con un'elevata correlazione reciproca (coefficiente di correlazione  $> 0.70$ ) per prevenire problemi di multicollinearità nel modello finale. Come si può osservare dalla **Matrice di correlazione** (Figura 1), c'è una coppia di feature fortemente correlate: **CrimeCode\_encoded** e **Description\_encoded**, con coefficiente di correlazione uguale

a circa **0.729**. Si decide quindi di **eliminare** la feature **CrimeCode\_encoded**, al fine di non influenzare la classificazione finale.

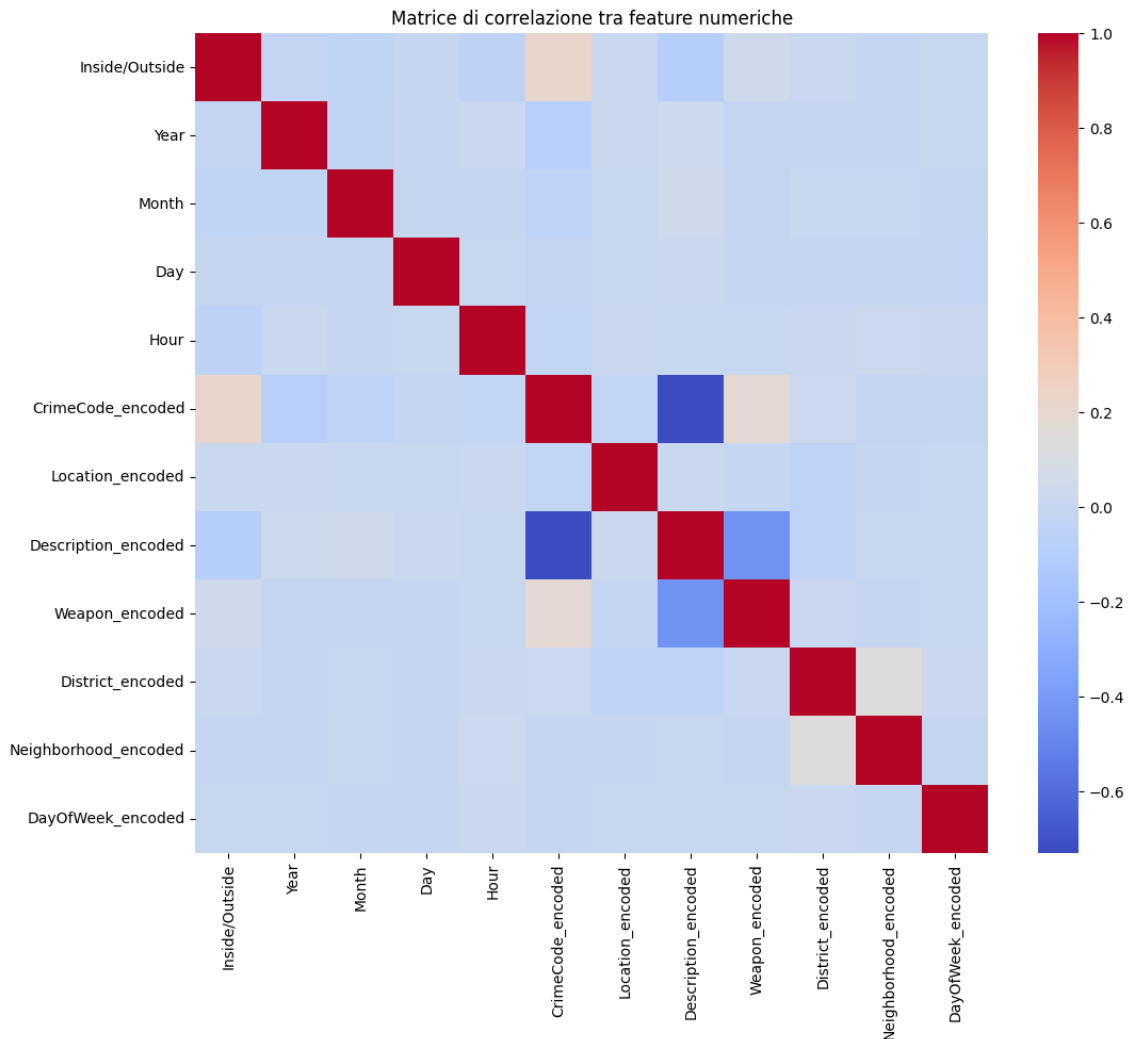


Figura 1: Matrice di correlazione tra feature numeriche

Si procede con la suddivisione del dataset finale in training (80%) e test (20%).

```

1 # Eliminazione delle feature fortemente correlate
2 crime_data_weapon = crime_data_weapon.drop(columns=['CrimeCode_encoded
  '])
3
4 X = crime_data_weapon.drop('Weapon_encoded', axis=1)
5 y = crime_data_weapon['Weapon_encoded']
6
7 # Split del dataset in train e test
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, random_state=42, stratify=y

```

10 )

Osservando il numero di elementi per ogni classe di **Weapon\_encoded** e la loro distribuzione, si ottiene il seguente output:

```
1 --- Distribuzione delle classi nel dataset di training ---
2 Weapon_encoded
3 1 55.472387
4 0 18.494373
5 3 15.480167
6 2 10.553074
7 Name: proportion, dtype: float64
8
9 --- Numero di elementi per ogni classe nel dataset di training ---
10 Weapon_encoded
11 1 41353
12 0 13787
13 3 11540
14 2 7867
15 Name: count, dtype: int64
16
17 --- Distribuzione delle classi nel dataset di test ---
18 Weapon_encoded
19 1 55.470301
20 0 18.495466
21 3 15.479959
22 2 10.554274
23 Name: proportion, dtype: float64
24
25 --- Numero di elementi per ogni classe nel dataset di test ---
26 Weapon_encoded
27 1 10338
28 0 3447
29 3 2885
30 2 1967
31 Name: count, dtype: int64
```

Listing 1: Distribuzione delle classi di **Weapon\_encoded** nei dataset di training e test.

L'analisi evidenzia una predominanza della **Classe 1 (*HANDS*)**, che rappresenta circa il 55% delle osservazioni sia nel dataset di training sia in quello di test. Le altre classi risultano significativamente meno rappresentate, indicando uno **sbilanciamento** nella distribuzione delle categorie. Si osserva, inoltre, una buona coerenza tra training e test set, in quanto le proporzioni relative delle classi sono sostanzial-



mente simili, garantendo che il campionamento preservi la struttura originale dei dati.

Tale sbilanciamento **potrebbe influenzare negativamente** le prestazioni di modelli predittivi standard, poiché il modello potrebbe tendere a privilegiare la classe più frequente.

## 2.2 Training

Per affrontare il task di Classificazione è stato impiegato un *ensemble* diversificato di algoritmi di *Machine Learning*. La scelta di includere modelli sia semplici che complessi mira a valutare quale approccio garantisca il **miglior equilibrio** tra performance e capacità di generalizzazione sui dati.

Nel seguente listato vengono descritti i classificatori utilizzati per la fase di training.

```
1 classifiers = [  
2     ("DecisionTreeClassifier", DecisionTreeClassifier(random_state=42)  
3         , {"max_depth": [10,13,15,18, 20, 23]}),  
4     ("RandomForestClassifier", RandomForestClassifier(random_state=42)  
5         , {"n_estimators": [5,10,15, 25, 30], "max_depth":  
6             [10,13,15,18, 20, 23]}),  
7     ("AdaBoostClassifier", AdaBoostClassifier(random_state=42), {"  
8         n_estimators": [5,10,15, 25, 30]}),  
9     ("GradientBoostingClassifier", GradientBoostingClassifier(  
10         random_state=42), {"n_estimators": [5,15,25, 25, 30]}),  
11     ("NeuralNetwork", Pipeline([  
12         ('scaler', StandardScaler()),  
13         ('clf', MLPClassifier(  
14             hidden_layer_sizes=(64, 32, 16),  
15             activation='relu',  
16             solver='adam',  
17             max_iter=100,  
18             random_state=42  
19         ))  
20     ]), {"clf__hidden_layer_sizes": [(64,32,16),(32,16,8)], "  
21         clf__max_iter": [300,400]})  
22 ]
```

Verrà utilizzato un modello di **GridSearchCV** per trovare automaticamente la combinazione ottimale di iperparametri, attraverso una **cross-validation**. Ver-

ranno testati **6 alberi decisionali diversi**, ognuno con una profondità diversa ed usando **4-fold cross validation** per stimare l'Accuracy media di ogni figurazione.

## 2.3 Risultati della Classificazione

Di seguito, sono riportati i risultati della valutazione, utilizzando metriche di classificazione di **Accuracy**, **Precision**, **Recall**, **F1-Score** (in particolare il *macro avg* che valuta l'equilibrio tra classi) e l'**Area Under the Curve (AUC)**.

Tabella 2: Metriche di valutazione dei modelli di classificazione sul dataset.

| Modello            | Accuracy | Precision (Macro Avg) | Recall (Macro Avg) | F1-Score (Macro Avg) |
|--------------------|----------|-----------------------|--------------------|----------------------|
| Random Forest      | 0.796    | 0.655                 | 0.672              | 0.652                |
| Gradient Boosting  | 0.794    | 0.627                 | 0.655              | 0.631                |
| Ada Boost          | 0.783    | 0.628                 | 0.617              | 0.617                |
| Decision Tree      | 0.761    | 0.589                 | 0.598              | 0.593                |
| Rete Neurale (MLP) | 0.755    | 0.559                 | 0.589              | 0.569                |

Come si può osservare dalla Tabella 2, il **Random Forest** ha ottenuto le performance migliori in tutte le metriche aggregate. Seguono **GradientBoosting**, **NeuralNetwork** e **DecisionTree**, che si comportano in modo molto simile, con prestazioni comunque buone. Il modello **AdaBoostClassifier** ha prestazioni decisamente inferiori, con F1-Score particolarmente basso, indicando che fa molta fatica a bilanciare Precision e Recall.

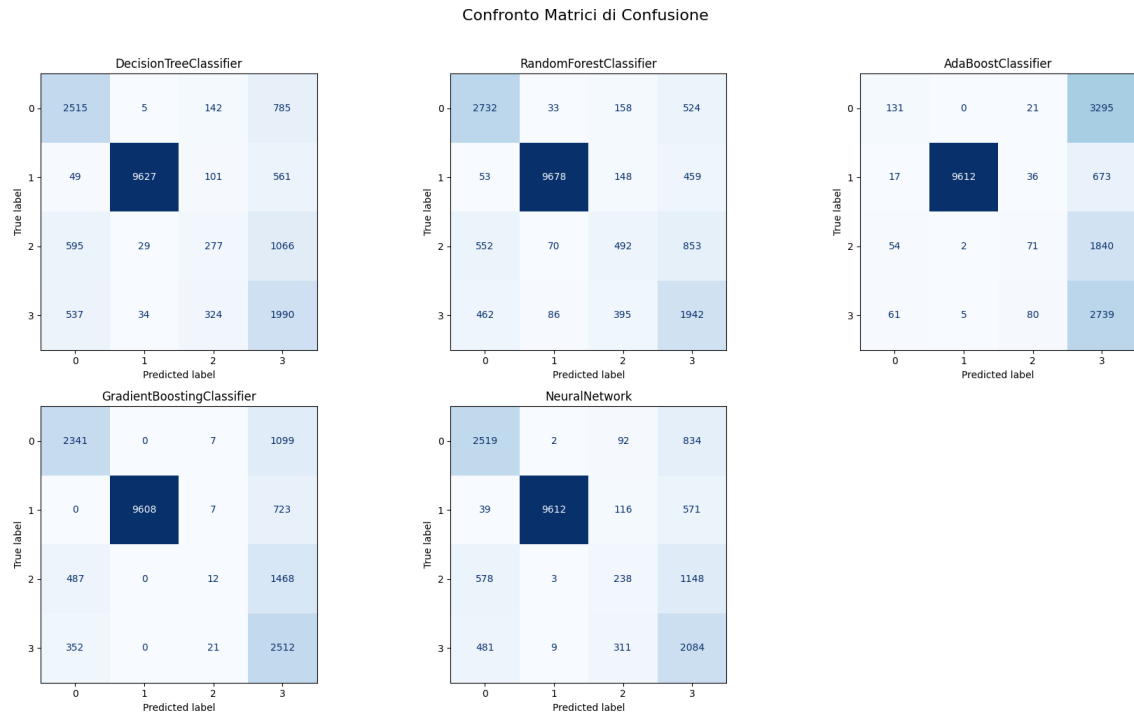


Figura 2: Confronto matrici di Confusione di ciascun modello

Analizzando le Matrici di confusione (Figura 2), emergono le sfide più interessanti: il problema principale è il **forte squilibrio delle classi**. Questo si può capire anche dal confronto tra Accuracy e F1-Score, dove l'Accuracy è alta, mentre l'F1-Score è più basso. Questa discrepanza suggerisce che l'Accuracy è "gonfiata" dalla capacità del modello di **indovinare correttamente la classe maggioritaria**. Infatti, osservando le matrici di confusione, si può notare come **tutti i modelli sono eccellenti nel predire la Classe 1**. Le **Classi 0 e 2** vengono spesso confuse ed erroneamente classificate come **Classe 3**, mentre i risultati del modello AdaBoost è visibile anche nella sua matrice di confusione, spiegando le metriche peggiori.

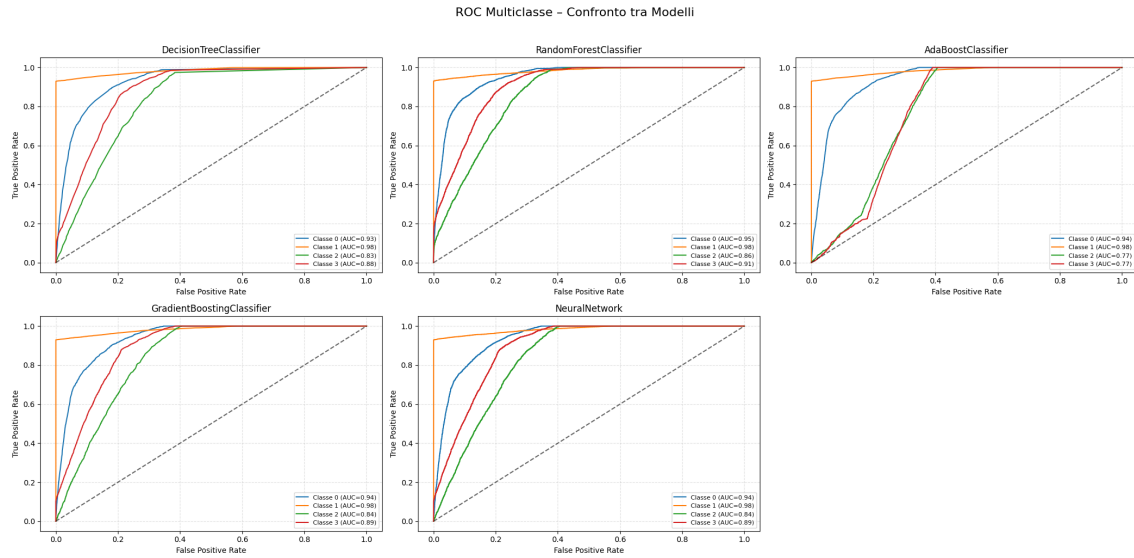


Figura 3: Confronto della curva ROC di ogni modello

Le curve ROC (Figura 3), mostrano l'abilità del modello di distinguere tra le classi.

- La **performance è buona**: le aree sotto la curva (AUC) sono molto alte per quasi tutti i modelli (escluso AdaBoost).
- Il **controsenso apparente**: un AUC alto significa che il modello assegna probabilità "corrette" (ad esempio un campione di Classe 0 ha una probabilità più alta di essere Classe 0 che di essere Classe 3). Tuttavia, quando si passa dalle probabilità alla classificazione finale, il modello "sceglie" la classe che massimizza la probabilità. A causa dello squilibrio, il modello si "sbilancia" preferendo la Classe 3 (o la 1) anche quando la probabilità per la Classe 0 o 2 è discreta. In breve, i modelli **sanno distinguere le classi** (alto AUC), ma **fanno la scelta sbagliata** al momento della classificazione finale a causa dello squilibrio dei dati.

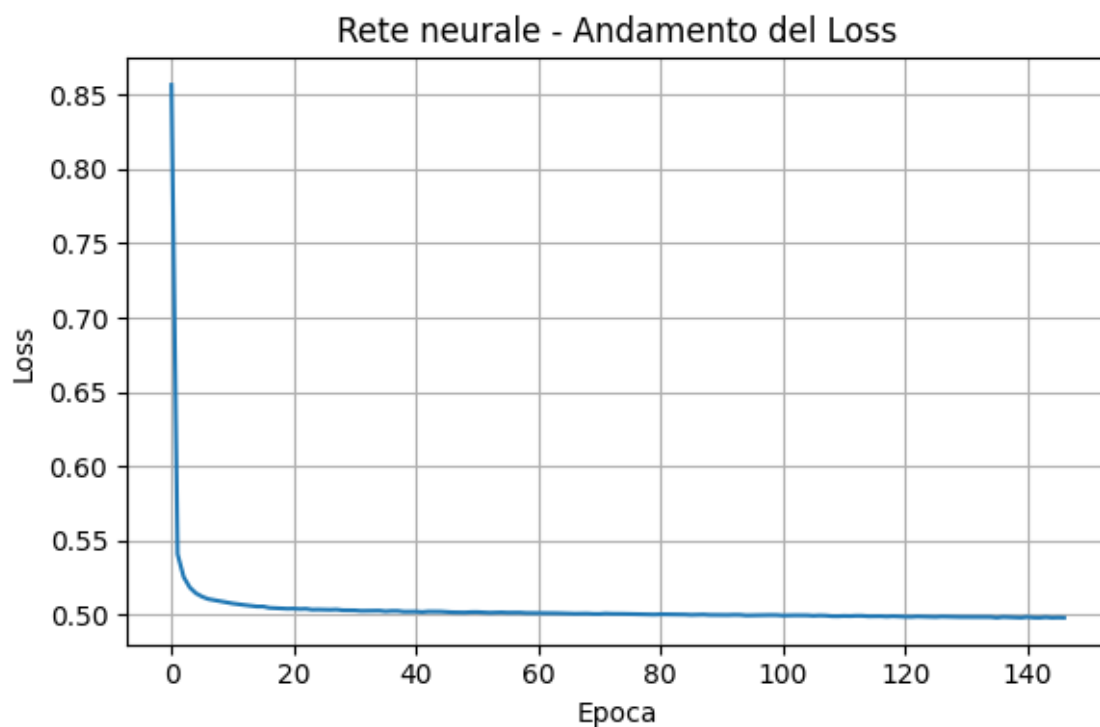


Figura 4: Andamento della Loss della rete neurale

Per quanto riguarda l'andamento della Loss della **Rete Neurale** (Figura 4), si può notare che:

- **Convergenza rapida:** l'errore scende verticalmente nelle prime 10-20 epoche, per poi stabilizzarsi intorno ad un valore di **0.50**.
- **Stabilità:** la curva si appiattisce e rimane bassa, indicando che il modello ha trovato una soluzione stabile e non sta "oscillando" o peggiorando (quindi overfitting) nelle epoche successive.

Inoltre, tutti i modelli sono stati **estremamente veloci** (poche decine di secondi al massimo) nell'addestramento. L'unica eccezione è la **Rete Neurale** che, pur essendo un modello avanzato, ha richiesto **tempi di addestramento significativamente maggiori (fino a 5 minuti)**.

In conclusione, il vero problema del dataset è quindi l'**impatto dello squilibrio delle classi**. L'Accuracy è alta, ma le prestazioni sulle classi minoritarie sono scarse.

## 2.4 Classificazione con SMOTEENN

Dal momento che le predizioni risultano discrete per la classe maggioritaria (*HANDS*), ma piuttosto povere per le restanti classi, si tenta l'implementazione di una tecnica

ibrida **Upsampling + Downsampling**: si intende aumentare artificialmente il numero di campioni presenti nel dataset di training, inerenti alle classi 0 ('FIREARM'), 2 ('KNIFE') e 3 ('OTHER') ed effettuarne poi una pulizia.

Si utilizzerà la tecnica **SMOTE+ENN** sul training set per creare "nuovi" campioni delle classi minoritarie. Usando una **Pipeline** viene assicurato che il ricampionamento avvenga solo sul set di addestramento all'interno di ogni singolo fold della cross-validation, evitando di "contaminare" i dati di validazione.

```
1 classifiers_smoteenn = [  
2     ("DecisionTreeClassifier",  
3     ImbPipeline([  
4         ('resampling', SMOTEENN(random_state=42)),  
5         ('model', DecisionTreeClassifier(random_state=42))  
6     ]),  
7     {"model__max_depth": [10, 13, 15, 18, 20, 23]}},  
8     ("RandomForestClassifier",  
9     ImbPipeline([  
10        ('resampling', SMOTEENN(random_state=42)),  
11        ('model', RandomForestClassifier(random_state=42))  
12    ]),  
13    {"model__n_estimators": [5, 10, 15, 25, 30], "model__max_depth":  
14        [10, 13, 15, 18, 20, 23]}},  
15    ("AdaBoostClassifier",  
16    ImbPipeline([  
17        ('resampling', SMOTEENN(random_state=42)),  
18        ('model', AdaBoostClassifier(random_state=42))  
19    ]),  
20    {"model__n_estimators": [5, 10, 15, 25, 30]}},  
21    ("GradientBoostingClassifier",  
22    ImbPipeline([  
23        ('resampling', SMOTEENN(random_state=42)),  
24        ('model', GradientBoostingClassifier(random_state=42))  
25    ]),  
26    {"model__n_estimators": [5, 15, 25, 25, 30]}},  
27    ("NeuralNetwork",  
28    ImbPipeline([  
29        ('resampling', SMOTEENN(random_state=42)),  
30        ('model', Pipeline([  
31            ('scaler', StandardScaler()),  
            ('clf', MLPClassifier(  

```

```

32         hidden_layer_sizes=(64, 32, 16),
33         activation='relu',
34         solver='adam',
35         max_iter=100,
36         random_state=42
37     ))
38 ]))
39 ],
40 {"model__clf__hidden_layer_sizes": [(64, 32, 16), (32, 16, 8)],
41  "model__clf__max_iter": [300, 400]})
42 ]

```

Come prima, verrà utilizzato il modello **GridSearchCV**, con gli stessi iperparametri.

## 2.5 Risultati della Classificazione con SMOTEENN

Di seguito, si riportano i risultati della seconda valutazione, utilizzando le stesse metriche di classificazione e si osserverà se la tecnica ibrida di Upsampling+Downsampling utilizzata ha effettivamente **migliorato** i modelli.

Tabella 3: Confronto delle metriche di performance dei modelli di classificazione con SMOTEENN.

| Modello                      | Accuracy | Precision (Macro) | Recall (Macro) | F1-Score (Macro) |
|------------------------------|----------|-------------------|----------------|------------------|
| Decision Tree Classifier     | 0.760    | 0.613             | 0.623          | 0.616            |
| Random Forest Classifier     | 0.779    | 0.638             | 0.651          | 0.643            |
| AdaBoost Classifier          | 0.773    | 0.537             | 0.616          | 0.558            |
| Gradient Boosting Classifier | 0.763    | 0.621             | 0.628          | 0.622            |
| Rete Neurale (MLP)           | 0.761    | 0.612             | 0.625          | 0.616            |

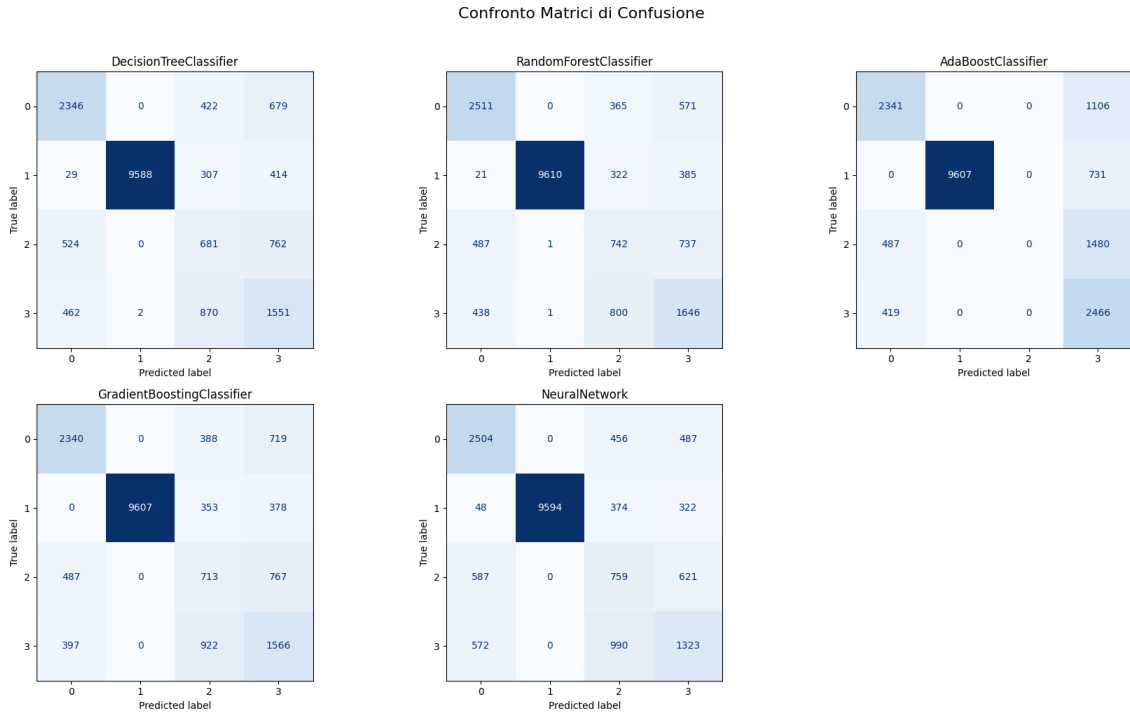


Figura 5: Confronto Matrici di Confusione con SMOTEENN

I risultati applicando la tecnica SMOTEENN sono contrastanti.

- Ai modelli che prima erano i peggiori, ora:
  - **AdaBoost**: Accuracy è cresciuta (da **0.673** a **0.773**) e F1-Score migliorato fino ad arrivare ad un modello competente (da **0.395** a **0.558**).
  - **GradientBoosting**: F1-Score migliorata (da **0.565** a **0.662**). Analizzando la matrice di confusione (Figura 5), si può notare come prima indovinava solo **12** campioni della Classe 2, ora ne indovina ben **713**, anche se è costato un po' di performance sulla Classe 3.
- Ai modelli che prima erano buoni, ora:
  - **RandomForest**: F1-Score peggiorato da **0.652** a **0.643**, Accuracy da **0.796** a **0.779**. Quindi, qualsiasi tentativo di bilanciamento ha peggiorato le prestazioni.



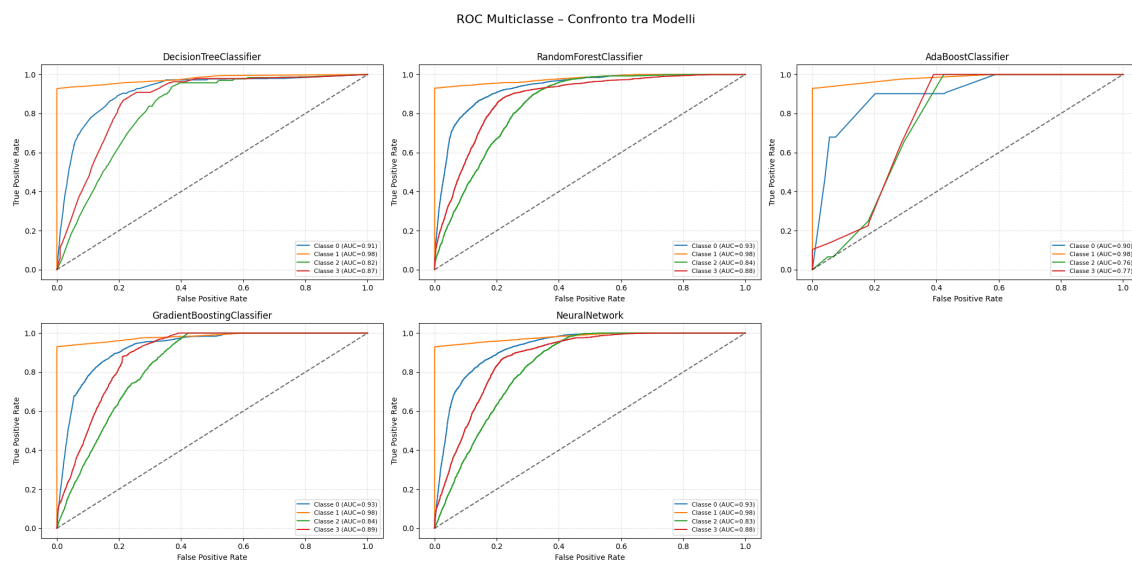


Figura 6: Confronto ROC con SMOTEENN

Osservando le **Curve ROC** (Figura 6), si può notare che per quasi tutti i modelli, i valori di **AUC** sono **scesi** rispetto al primo esperimento. Questo significa che i dati sintetici di SMOTE hanno reso i confini tra le classi più "confusi", rendendo più difficile per i modelli distinguere le classi.

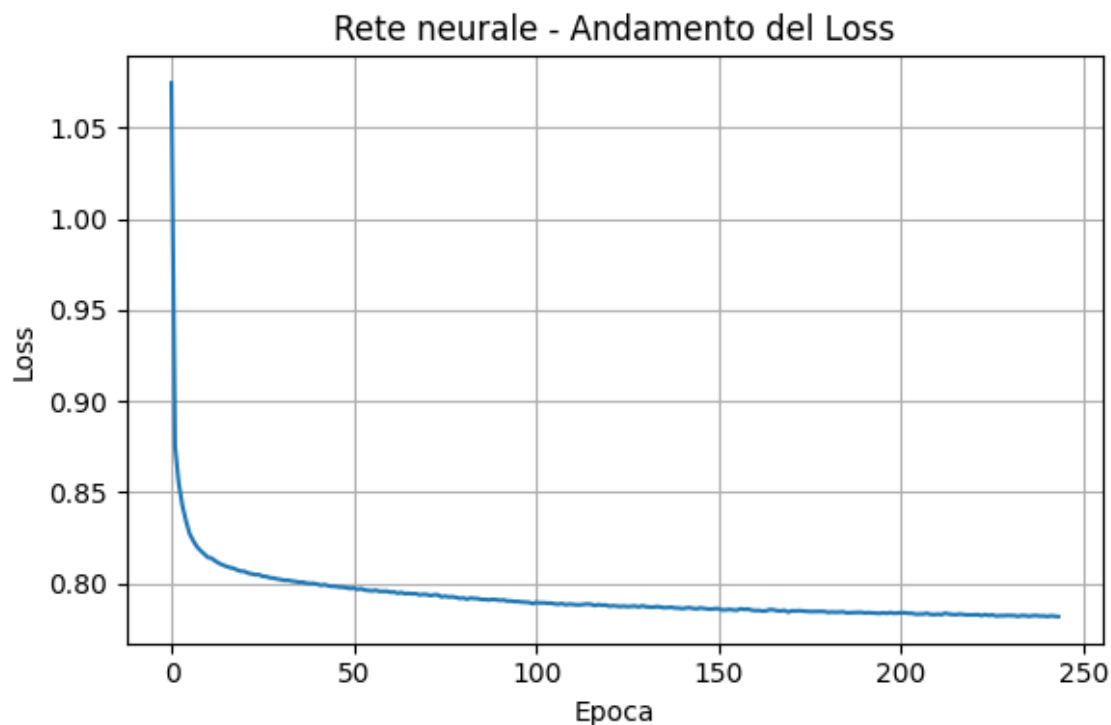


Figura 7: Andamento Loss con SMOTEENN

Anche la **Loss della rete neurale** (Figura 7) ora parte molto più alta e si

stabilizza più in alto (circa **0.78** contro **0.50**). Questo conferma che il nuovo dataset ricampionato è più rumoroso e più difficile da imparare per il modello.

## 2.6 Conclusioni e osservazioni

Dal punto di vista metodologico, i risultati ottenuti dimostrano come **la qualità del bilanciamento dei dati giochi un ruolo cruciale** nelle prestazioni dei modelli di classificazione. In particolare, tecniche di ricampionamento come **SMOTEENN**, pur riuscendo ad attenuare lo sbilanciamento tra le classi, possono introdurre **rumore e ambiguità nei confini decisionali**, riducendo la capacità del modello di generalizzare correttamente.

Il **Random Forest** si conferma il modello più robusto, grazie alla sua natura **ensemble** e alla capacità di ridurre la varianza aggregando molteplici alberi decisionali. Tale comportamento risulta particolarmente vantaggioso in contesti con dati rumorosi o distribuzioni non uniformi.

L'analisi delle curve ROC e delle matrici di confusione ha inoltre evidenziato che la maggior parte degli errori si concentra nelle classi minoritarie (*FIREARM* e *KNIFE*).

Infine, si evidenzia che, nonostante le buone performance complessive, l'accuratezza da sola non è un indicatore sufficiente per valutare modelli su dataset sbilanciati: metriche come **F1-Score macro** e **AUC** risultano più affidabili per misurare la reale capacità del modello di distinguere tra le classi.

## 3 Clustering

L'obiettivo dell'analisi è quello di effettuare una Clusterizzazione dei distretti di polizia basati sui loro profili di criminalità, utilizzando algoritmi di clustering come **K-Means**.

Per l'analisi verranno utilizzate librerie **Python** come **pandas**, **matplotlib**, **seaborn**, **numpy**, **scipy** e **sklearn**, che consentiranno di effettuare una **pulizia accurata dei dati**, un' **analisi esplorativa** e la **modellazione del modello di clustering**.

### 3.1 Preparazione dati

Il processo ETL per il Clustering mira a creare **profili statistici** per ciascun distretto. La trasformazione focalizza l'attenzione sui **pattern comportamentali e ambientali** del crimine:

- **Costruzione del profilo:** le variabili categoriali (tipo di reato, arma, fascia oraria e contesto **Inside/Outside**) sono state convertite in variabili binarie tramite **One-Hot Encoding**. Queste sono state poi **aggregate calcolando la media per ciascun distretto**, trasformando ogni riga in un profilo percentuale.
- **Standardizzazione:** per l'applicazione di algoritmi basati sulla distanza, i profili distrettuali sono stati **standardizzati** per assicurare che tutte le caratteristiche contribuissero in modo equo alla misurazione della distanza tra i distretti.

```
1 # Uniformizzazione stringhe
2 df['Description'] = df['Description'].str.strip()
3 df['District'] = df['District'].str.strip().replace({
4     'Central': 'CENTRAL',
5     'NORTHESTERN': 'NORTHEASTERN',
6     'SOUTHESTERN': 'SOUTHEASTERN',
7     'Gay Street': 'EASTERN'
8 })
9 # Standardizzazione piu' chiara di Inside/Outside
10 io_map = {'I': 'Inside', 'Inside': 'Inside', 'O': 'Outside', 'Outside': 'Outside'}
11 df['Inside/Outside'] = df['Inside/Outside'].map(io_map)
12
```

```

13 # Creazione fasce orarie
14 bins = [-0.1, 5, 11, 17, 23]
15 labs = ['Night', 'Morning', 'Afternoon', 'Evening']
16 df['HourBin'] = pd.cut(df['CrimeTime'], bins=bins, labels=labs)
17
18 # One-hot encoding (dummies)
19 dummies = pd.get_dummies(df[cat_cols], drop_first=False)

```

Il risultato finale è la suddivisione dei Distretti in **nove** zone differenti. La tabella è composta da valori che rappresentano una *percentuale*, con le seguenti distribuzioni per ogni colonna:

```

1 District Description_AGG. ASSAULT Description_COMMON ASSAULT Description_RAPE
  ↳ Description_ROBBERY-CARJACKING Description_ROBBERY-COMMERCIAL
  ↳ Description_ROBBERY-RESIDENCE Description_ROBBERY-STREET
  ↳ Weapon_FIREARM Weapon_HANDS Weapon_KNIFE Weapon_OTHER HourBin_Night
  ↳ HourBin_Morning HourBin_Afternoon HourBin_Evening Inside/
  ↳ Outside_Inside Inside/Outside_Outside
2 CENTRAL 0.259643 0.534182 0.018151 0.009470 0.028904 0.011147 0.138503
  ↳ 0.158528 0.572359 0.119463 0.149650 0.261221 0.136825 0.270100
  ↳ 0.331854 0.454474 0.545526
3 EASTERN 0.323434 0.546379 0.013934 0.007425 0.021766 0.009662 0.077400
  ↳ 0.158259 0.584215 0.114931 0.142596 0.211961 0.157140 0.288242
  ↳ 0.342657 0.539056 0.460944
4 NORTHEASTERN 0.274290 0.523088 0.015368 0.010998 0.037363 0.017698 0.121194
  ↳ 0.202185 0.565550 0.091988 0.140277 0.211945 0.149964 0.285943
  ↳ 0.352149 0.619374 0.380626
5 NORTHERN 0.272212 0.479088 0.018667 0.012760 0.049504 0.020085 0.147684
  ↳ 0.228852 0.519731 0.100425 0.150992 0.235113 0.142486 0.274102
  ↳ 0.348299 0.572543 0.427457
6 NORTHWESTERN 0.313379 0.455896 0.016667 0.017800 0.042290 0.015986 0.137982
  ↳ 0.243084 0.496372 0.103628 0.156916 0.199206 0.162245 0.281179
  ↳ 0.357370 0.556009 0.443991
7 SOUTHEASTERN 0.268477 0.540654 0.013201 0.007063 0.034810 0.011856 0.123938
  ↳ 0.157740 0.577062 0.115951 0.149247 0.289834 0.120659 0.232490
  ↳ 0.357017 0.500883 0.499117
8 SOUTHERN 0.316842 0.516364 0.016938 0.008325 0.030813 0.022871 0.087847
  ↳ 0.161722 0.565072 0.099426 0.173780 0.256459 0.134354 0.252057
  ↳ 0.357129 0.548900 0.451100
9 SOUTHWESTERN 0.335501 0.494385 0.016905 0.013595 0.028609 0.017969 0.093037
  ↳ 0.207117 0.537179 0.099184 0.156520 0.199669 0.157111 0.302045
  ↳ 0.341175 0.569334 0.430666
10 WESTERN 0.353855 0.505176 0.016079 0.012335 0.018502 0.011894 0.082159
  ↳ 0.172026 0.548128 0.108921 0.170925 0.194604 0.161123 0.306278
  ↳ 0.337996 0.536233 0.463767

```

---

Listing 2: Distribuzione relativa dei crimini, armi e fasce orarie per distretto.

### 3.2 Clustering dei distretti con K-Means

Dopo la costruzione dei profili percentuali di ciascun distretto, si passa alla fase di **Clustering** attraverso l'implementazione dell'algoritmo **K-Means**, con l'obiettivo di individuare gruppi di distretti che presentano similitudini nei pattern di criminalità.

K-Means permette di suddividere i distretti in un numero prefissato di cluster  $K$ , minimizzando la distanza dei punti dai rispettivi centroidi. In questa fase, vengono:

- Standardizzate le feature per rendere confrontabili le variabili (poiché K-Means si basa sulla distanza);
- Individuato i valori ottimali di  $K$  tramite i metodi **Elbow** e **Silhouette**;
- Assegnati i distretti ai cluster ottenuti.

Questa prima analisi fornisce una base quantitativa su cui poi poggeranno le successive fasi esplorative e interpretative (PCA, heatmap, Lift, ecc.) che mirano a comprendere come e perché i distretti si raggruppano in certi modi.

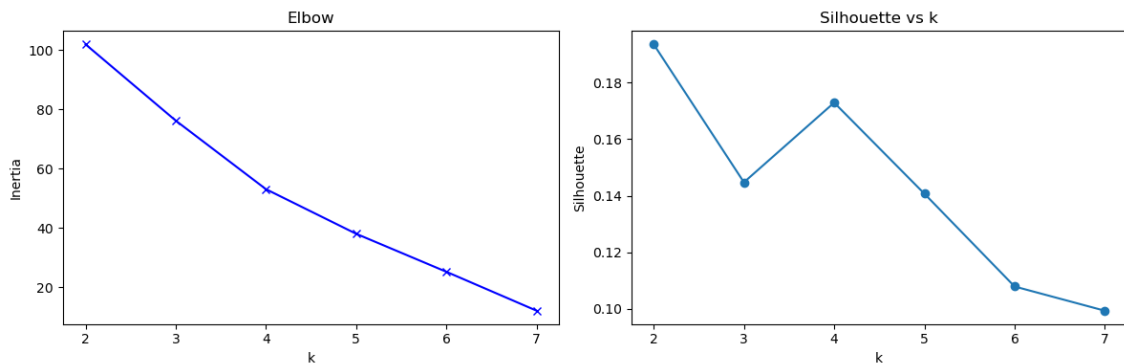


Figura 8: Grafici Elbow Method e Silhouette

Dai grafici (Figura 8) si stabilisce che **K=4** sia il valore ottimale di cluster. L'algoritmo K-Means viene eseguito sui profili di distretto standardizzati, assegnando a ciascun distretto un'etichetta di cluster (0,1,2 o 3).

```
1 # === Fit finale di KMeans e assegnazione ai distretti ===
2 k_opt = 4
3 kmeans = KMeans(n_clusters=k_opt, n_init='auto', random_state=0)
```

```

4 labels = kmeans.fit_predict(X_scaled)
5
6 # Assegnazione cluster ai distretti
7 clustered = agg_df.copy()
8 clustered['cluster'] = labels

```

L'algoritmo ha suddiviso i distretti in **quattro gruppi** distinti e bilanciati. Dopo la sua esecuzione, si possono visualizzare il numero dei distretti contenuti in ogni cluster e l'ordine dei distretti per ogni cluster (Tabella 4). Tuttavia, poiché K-Means assegna ogni osservazione a un cluster anche quando è un caso limite, non è in grado di identificare outlier o anomalie.

Tabella 4: Raggruppamento dei distretti per cluster

| Cluster | Numero distretti | Nomi distretti                 |
|---------|------------------|--------------------------------|
| 0       | 2                | NORTHERN, NORTHWESTERN         |
| 1       | 2                | CENTRAL, SOUTHEASTERN          |
| 2       | 2                | NORTHEASTERN, SOUTHERN         |
| 3       | 3                | EASTERN, SOUTHWESTERN, WESTERN |

### 3.3 Analisi visiva dei cluster tramite PCA

Prima di passare ad analisi più avanzate, si procede con una rappresentazione bidimensionale tramite **PCA (Principal Component Analysis)**, utile per comprendere visivamente la separazione tra i cluster.

In questa fase, non si creano nuovi cluster, ma si interpreta graficamente la struttura già calcolata da K-Means.

La **PCA** viene utilizzata per ridurre la dimensionalità dei profili dei distretti, consentendo una rappresentazione visiva dei cluster individuati da K-Means. Questa tecnica permette di:

- Proiettare i dati in un piano bidimensionale mantenendo la massima varianza possibile;
- Identificare le dimensioni latenti che spiegano la separazione tra i distretti;
- Verificare visivamente la coerenza e separazione dei cluster ottenuti.

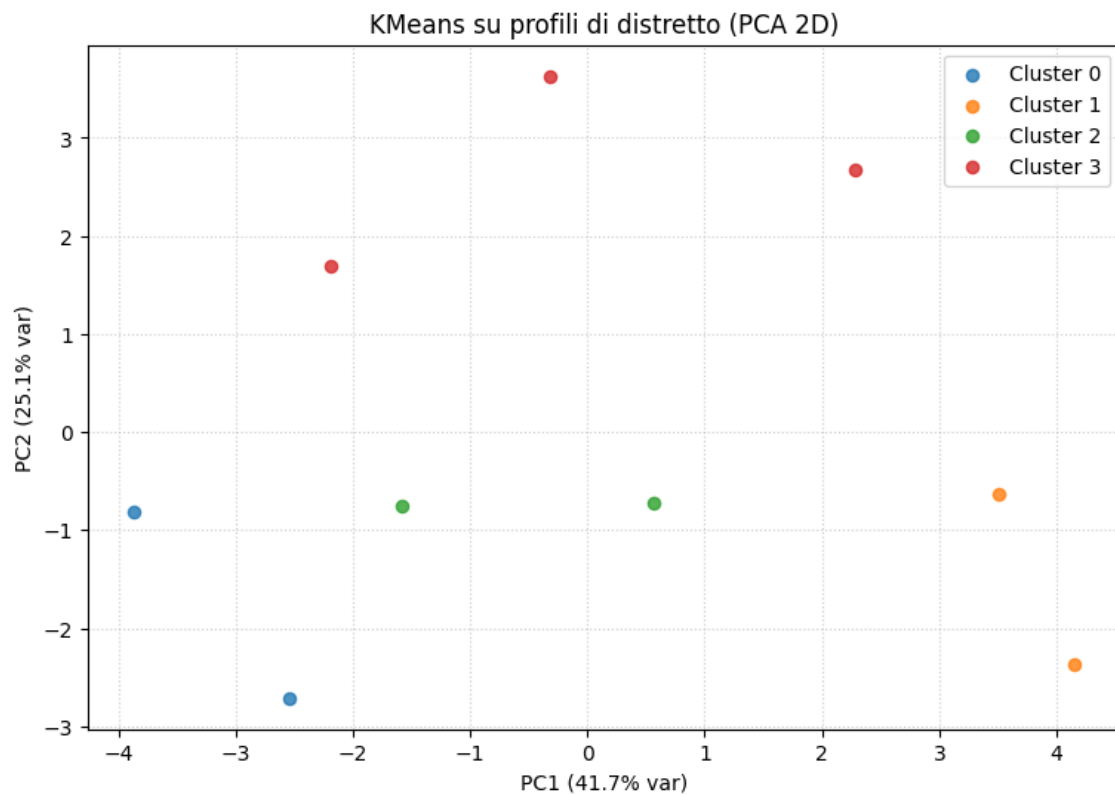


Figura 9: K-Means su profili di distretto - PCA

```

1  === PC1 (41.7%) - Cosa sposta i punti a destra/sinistra? ===
2  --- Spingono a DESTRA (Valori Alti) ---
3  Description_COMMON ASSAULT 0.332926
4  Weapon_HANDS 0.322216
5  Weapon_KNIFE 0.294408
6  Inside/Outside_Outside 0.283471
7  HourBin_Night 0.268611
8  Name: PC1, dtype: float64
9
10 --- Spingono a SINISTRA (Valori Bassi) ---
11 Weapon_FIREARM -0.342732
12 Description_ROBBERY - CARJACKING -0.327928
13 Inside/Outside_Inside -0.283471
14 HourBin_Morning -0.242137
15 Description_ROBBERY - RESIDENCE -0.222155
16 Name: PC1, dtype: float64
17
18 === PC2 (25.1%) - Cosa sposta i punti in alto/basso? ===
19 --- Spingono in ALTO (Valori Alti) ---
20 Description_AGG. ASSAULT 0.399974
21 HourBin_Afternoon 0.360607
22 HourBin_Morning 0.332476

```

```

23 Weapon_OTHER 0.122593
24 Weapon_HANDS 0.083462
25 Name: PC2, dtype: float64
26
27 --- Spingono in BASSO (Valori Bassi) ---
28 Description_ROBBERY - COMMERCIAL -0.415056
29 Description_ROBBERY - STREET -0.392394
30 HourBin_Night -0.316152
31 HourBin_Evening -0.270515
32 Description_ROBBERY - RESIDENCE -0.200044
33 Name: PC2, dtype: float64

```

Listing 3: Stampa delle feature più influenti per PC1 (Asse X) e PC2 (Asse Y)

Dal grafico della PCA (Figura 9), si nota chiaramente che i cluster formano gruppi distinti, ben separati e significativi. Inoltre, osservando il Listato 3, si può osservare come:

- L'asse X, che rappresenta PC1 (41.7% della varianza), crea una separazione basata su **gravità del crimine e luogo**:
  - **Valori positivi (a destra)** dominati da: *Description\_COMMON ASSAULT*, *Weapon\_HANDS*, *Weapon\_KNIFE*, *Inside/Outside\_Outside* e *HourBin\_Night*. I distretti a destra del grafico sono caratterizzati da **crimini meno gravi, commessi all'esterno, spesso di notte e senza armi da fuoco**.
  - **Valori negativi (a sinistra)** dominati da: *Weapon\_FIREARM*, *ROBBERY - CARJACKING*, *Inside/Outside\_Inside* e *ROBBERY - RESIDENCE*. I distretti a sinistra del grafico sono caratterizzati da **crimini più gravi, con armi da fuoco, spesso legati a rapine e che avvengono all'interno**.
- L'asse Y, che rappresenta PC2 (23.5% della varianza), riflette una distinzione basata su **momento della giornata e tipologia di crimine**:
  - **Valori positivi (in alto)** dominati da: *Description\_AGG. ASSAULT*, *HourBin\_Afternoon* e *HourBin\_Morning*. I distretti in alto nel grafico presentano una maggiore incidenza di **aggressioni (sia comuni che aggravate) che avvengono di giorno**.
  - **Valori negativi (in basso)** dominati da: *ROBBERY - COMMERCIAL*, *ROBBERY - STREET*, *HourBin\_Night* e *HourBin\_Evening*. I



distretti in basso nel grafico presentano una maggiore incidenza di **rapine** (commerciali e in strada) che avvengono di sera e di notte.

### 3.4 Interpretazione dei centroidi

Dopo le analisi visive si analizzano i centroidi, ovvero i **profili medi** dei distretti appartenenti a ciascun gruppo. L'obiettivo è comprendere quali caratteristiche definiscono ciascun cluster: quali tipologie di crimine, orari o contesti (*Inside/Outside*) sono predominanti.

Questo passo fornisce una prima interpretazione qualitativa, utile per delineare la "personalità" di ogni cluster, prima di passare ad analisi più quantitative.

```
1 centers_unscaled = pd.DataFrame(  
2     scaler.inverse_transform(kmeans.cluster_centers_),  
3     columns=agg_df.columns,  
4     index=[f'Cluster {i}' for i in range(k_opt)]  
5 )
```

```
1 === Cluster 0: feature dominanti ===  
2 Inside/Outside_Inside 0.564  
3 Weapon_HANDS 0.508  
4 Description_COMMON ASSAULT 0.467  
5 Inside/Outside_Outside 0.436  
6 HourBin_Evening 0.353  
7 Description_AGG. ASSAULT 0.293  
8 HourBin_Afternoon 0.278  
9 Weapon_FIREARM 0.236  
10 HourBin_Night 0.217  
11 Weapon_OTHER 0.154  
12 Name: Cluster 0, dtype: float64  
13  
14 === Cluster 1: feature dominanti ===  
15 Weapon_HANDS 0.575  
16 Description_COMMON ASSAULT 0.537  
17 Inside/Outside_Outside 0.522  
18 Inside/Outside_Inside 0.478  
19 HourBin_Evening 0.344  
20 HourBin_Night 0.276  
21 Description_AGG. ASSAULT 0.264  
22 HourBin_Afternoon 0.251  
23 Weapon_FIREARM 0.158  
24 Weapon_OTHER 0.149  
25 Name: Cluster 1, dtype: float64  
26
```

```

27 === Cluster 2: feature dominanti ===
28 Inside/Outside_Inside 0.584
29 Weapon_HANDS 0.565
30 Description_COMMON ASSAULT 0.520
31 Inside/Outside_Outside 0.416
32 HourBin_Evening 0.355
33 Description_AGG. ASSAULT 0.296
34 HourBin_Afternoon 0.269
35 HourBin_Night 0.234
36 Weapon_FIREARM 0.182
37 Weapon_OTHER 0.157
38 Name: Cluster 2, dtype: float64
39
40 === Cluster 3: feature dominanti ===
41 Weapon_HANDS 0.557
42 Inside/Outside_Inside 0.548
43 Description_COMMON ASSAULT 0.515
44 Inside/Outside_Outside 0.452
45 HourBin_Evening 0.341
46 Description_AGG. ASSAULT 0.338
47 HourBin_Afternoon 0.299
48 HourBin_Night 0.202
49 Weapon_FIREARM 0.179
50 HourBin_Morning 0.158
51 Name: Cluster 3, dtype: float64

```

Listing 4: Feature dominanti dei centroidi

L'output mostra la "personalità" di ogni cluster, elencando le 10 caratteristiche dominanti per ogni gruppo di distretti. Si può notare come *Weapons\_HANDS* e *Description\_COMMON ASSAULT* appaiono come le prime feature dominanti per tutti i cluster. Ciò non significa che i cluster siano tutti uguali, ma che questi sono i tipi di crimine/arma più comuni in tutta la città. Inoltre, si può notare come nel Cluster 2 e 0 la feature dominante è *Inside/Outside\_Inside*, suggerendo che la feature è stata molto significativa per la clusterizzazione.

Per confrontare in modo immediato i profili medi dei cluster, viene costruita una heatmap delle percentuali di ciascuna feature. Il colore indica la frequenza relativa: le celle più chiare rappresentano feature più comuni. Questa visualizzazione permette di:

- Individuare le feature condivise da tutti i cluster (es. crimini più diffusi in tutta la città).

- Riconoscere i tratti distintivi di ciascun gruppo (es. armi, orari, contesto interno/esterno).
- Identificare le feature rare, presenti in misura minima su tutti i distretti.

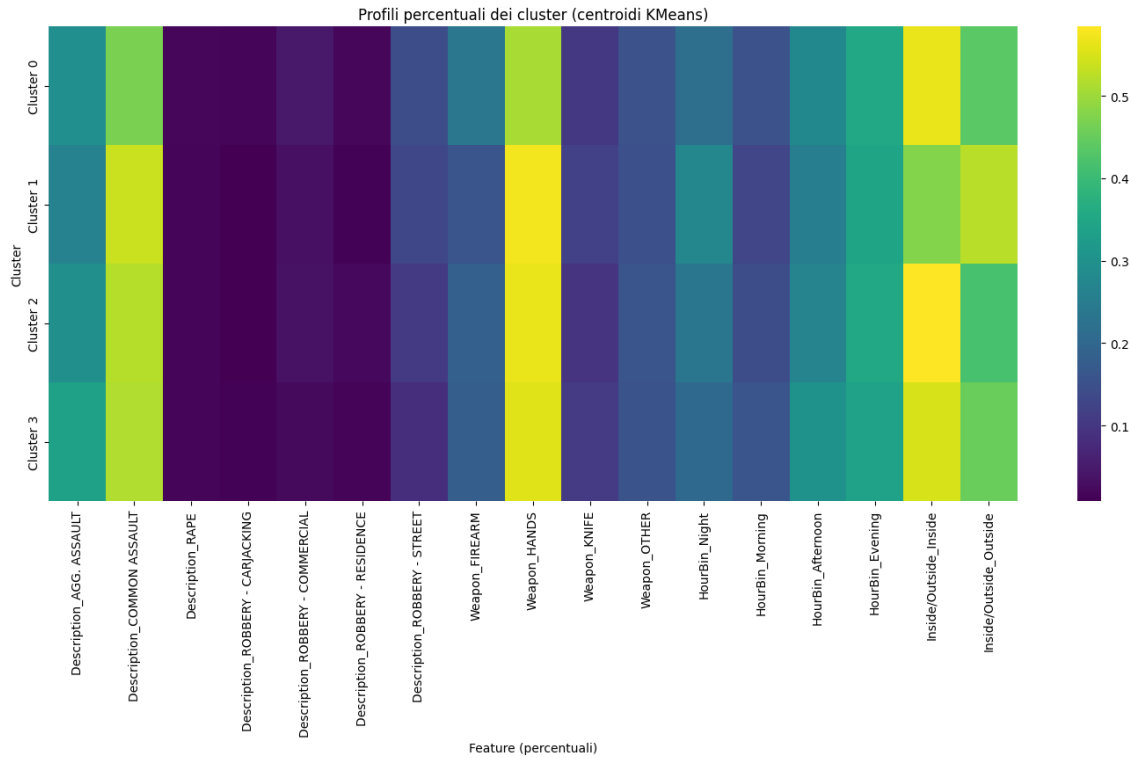


Figura 10: Profili percentuali dei Cluster

Come già visto in precedenza e come ci si aspettava, in Figura 10 si può notare come le colonne *Weapons\_HANDS*, *Description\_COMMON ASSAULT*, *Inside/Outside\_Outside* e *Inside/Outside\_Inside* sono le più luminose (dal verde al giallo) su tutte e 4 le righe.

Si osservano, di seguito, le differenze tra i cluster:

- **Cluster 1:** ha la cella più luminosa (giallo brillante) in assoluto per *Inside/Outside\_Inside*, ed è anche la riga più luminosa per *HourBin\_Night*. Questo cluster è chiaramente definito da crimini che avvengono **all'esterno e di notte**.
- **Cluster 0:** è visibilmente più luminoso (giallo-verde) su *Weapons\_FIREARM*. Questo cluster ha un profilo unico con una percentuale significativamente più alta di crimini con **armi da fuoco**, che avvengono prevalentemente **all'esterno**.

- **Cluster 3:** è la riga più luminosa (verde più brillante) sulla colonna *Description\_AGG. ASSAULT*, descrivendo come è un cluster di crimini che avvengono **all'interno** e caratterizzato da **aggressioni aggravate**.
- **Cluster 2:** sembra un profilo medio o generico di crimini *Inside/Outside\_Inside*. Ha alte percentuali di *COMMON ASSAULT* e *Weapon\_HANDS*, ma senza i picchi distintivi visti nel Cluster 1, 0 o 3.

Ci sono molte feature rare ovunque (colonne come *Description\_RAPE*, *Description\_HOMICIDE*, *Description\_ROBBERY - CARJACKING* e anche *Description\_SHOOTING*, che sono viola scuro per tutti i cluster, indicando che sono eventi rari (con percentuali basse) in tutti i distretti.

### 3.5 Analisi quantitativa tramite Lift

Dopo la fase esplorativa e qualitativa, si passa ora a un'analisi più quantitativa e comparativa. L'obiettivo è misurare, per ciascun cluster, **quanto ogni caratteristica è più o meno rappresentata** rispetto alla media generale del dataset. Il lift quantifica questa differenza, permettendo di:

- Individuare le feature sovra-rappresentate ( $\text{Lift} > 1$ ), sotto-rappresentate ( $\text{Lift} < 1$ ) o in linea con la media ( $\text{Lift} = 1$ );
- Cogliere variazioni locali che la heatmap non mostrava visivamente;
- Attribuire una vera e propria "personalità statistica" ai cluster.

In sintesi, mentre la heatmap mostra quanto una feature è frequente, il Lift mostra quanto più del normale lo è in ciascun cluster.

```

1 # === LIFT rispetto alla media globale ===
2 global_mean = agg_df.mean()
3
4 lift = centers_unscaled.copy()
5 for col in lift.columns:
6     baseline = global_mean[col] if global_mean[col] != 0 else 1e-9
7     lift[col] = lift[col] / baseline # >1 = sopra la media globale
8
9 for c in lift.index:
10     print(f'\n=== {c}: feature con lift pi alto ===')
11     print(
12         lift.loc[c]
```

```

13         .sort_values(ascending=False)
14         .head(10)
15         .round(2))

```

```

1  === Cluster 0: feature con lift pi alto ===
2  Description_ROBBERY - COMMERCIAL 1.41
3  Description_ROBBERY - CARJACKING 1.38
4  Description_ROBBERY - STREET 1.27
5  Weapon_FIREARM 1.26
6  Description_ROBBERY - RESIDENCE 1.17
7  Description Rape 1.09
8  HourBin_Morning 1.04
9  Inside/Outside_Inside 1.04
10 HourBin_Evening 1.02
11 HourBin_Afternoon 1.00
12 Name: Cluster 0, dtype: float64
13
14  === Cluster 1: feature con lift pi alto ===
15 HourBin_Night 1.20
16 Description_ROBBERY - STREET 1.17
17 Inside/Outside_Outside 1.15
18 Weapon_KNIFE 1.11
19 Description_COMMON ASSAULT 1.05
20 Weapon_HANDS 1.04
21 HourBin_Evening 0.99
22 Description_ROBBERY - COMMERCIAL 0.98
23 Weapon_OTHER 0.97
24 Description Rape 0.97
25 Name: Cluster 1, dtype: float64
26
27  === Cluster 2: feature con lift pi alto ===
28 Description_ROBBERY - RESIDENCE 1.31
29 Inside/Outside_Inside 1.07
30 Description_ROBBERY - COMMERCIAL 1.05
31 Weapon_HANDS 1.02
32 HourBin_Night 1.02
33 HourBin_Evening 1.02
34 Description_COMMON ASSAULT 1.02
35 Weapon_OTHER 1.02
36 Description Rape 1.00
37 Description_AGG. ASSAULT 0.98
38 Name: Cluster 2, dtype: float64
39
40  === Cluster 3: feature con lift pi alto ===
41 Description_AGG. ASSAULT 1.12

```

```

42 HourBin_Afternoon 1.08
43 HourBin_Morning 1.08
44 Weapon_KNIFE 1.02
45 Weapon_OTHER 1.01
46 Description_COMMON ASSAULT 1.01
47 Weapon_HANDS 1.01
48 Inside/Outside_Inside 1.01
49 Description_ROBBERY - CARJACKING 1.00
50 Inside/Outside_Outside 0.99
51 Name: Cluster 3, dtype: float64

```

Listing 5: Feature con Lift più alti

Osservando il Listing 5, si può ottenere un profilo statistico più accurato per ognuno dei cluster:

- **Cluster 0 (Distretti *NORTHERN* e *NORTHWESTERN*):** è il cluster delle rapine con armi da fuoco. Le caratteristiche distintive sono: *ROBBERY - COMMERCIAL* (1.41x), *ROBBERY - CARJACKING* (1.38x), *ROBBERY - STREET* (1.27x) e *Weapon\_FIREARM* (1.26x). Questo cluster rappresenta aree dove i crimini violenti armati sono significativamente più frequenti rispetto alla media cittadina.
- **Cluster 1 (Distretti *CENTRAL* e *SOUTHEASTERN*):** è il cluster dei crimini notturni e all'esterno, con *HourBin\_Night* (1.20x), *ROBBERY - STREET* (1.17x) e *Inside/Outside\_Outside* (1.15x). Anche le armi da taglio (*Weapon\_KNIFE*, 1.11x) risultano più presenti rispetto alla media.
- **Cluster 2 (Distretti *NORTHEASTERN* e *SOUTHERN*):** è il cluster delle rapine a proprietà e all'interno, definito da *ROBBERY - RESIDENCE* (1.31x) e *ROBBERY - COMMERCIAL* (1.05x). Si tratta di aree dove prevalgono reati di intrusione o a danno di esercizi, spesso in contesti interni.
- **Cluster 3 (Distretti *EASTERN*, *SOUTHWESTERN* e *WESTERN*):** è il cluster delle aggressioni diurne, con *AGG. ASSAULT* (1.12x), *HourBin\_Afternoon* (1.08x) e *HourBin\_Morning* (1.08x). Indica una maggiore concentrazione di aggressioni fisiche durante le ore diurne rispetto alla media complessiva.

In sintesi, il confronto con la media generale tramite Lift conferma e raffina quanto osservato nella heatmap: i Cluster 0 e 1 rappresentano contesti violenti o notturni, con un'elevata incidenza di armi o crimini all'esterno; i cluster 2 e 3

mostrano invece profili più legati all'interno o a situazioni di aggressione fisica, spesso in fasce orarie diurne o pomeridiane.

L'analisi del Lift si dimostra quindi un passaggio chiave per quantificare la rilevanza relativa delle feature nei vari cluster, permettendo di interpretare i risultati non solo in termini qualitativi (colore della heatmap) ma anche statistici (scostamento rispetto alla media).

### 3.6 Analisi gerarchica dei distretti

Per validare e approfondire ulteriormente i risultati precedenti, si esegue un **cluster gerarchico**. Questa tecnica fornisce una rappresentazione ad albero che evidenzia quali distretti sono più simili e in che ordine si aggregano, confermando (o ricalibrando) le conclusioni tratte dal K-Means. Infatti, questa tecnica:

- Consente di visualizzare la forza dei legami tra i gruppi identificati;
- Mostra progressivamente come i distretti si uniscono in base alla loro somiglianza;
- Non richiede di fissare a priori il numero di cluster.

```
1 # Calcola il linkage usando 'ward' (minimizza la varianza, simile a K-
   Means)
2 # Usa gli stessi dati standardizzati (X_scaled) usati per K-Means
3 Z = linkage(X_scaled, method='ward')
4
5 # Plotta il dendrogramma
6 plt.figure(figsize=(12, 6))
7 dendrogram(
8     Z,
9     labels=agg_df.index,
10    leaf_rotation=90,
11    leaf_font_size=10
12 )
13 plt.title('Dendrogramma Clustering Gerarchico dei Distretti')
14 plt.ylabel('Distanza (dissimilarit)')
15 plt.grid(axis='y', linestyle=':', alpha=0.7)
16 plt.tight_layout()
17 plt.show()
```

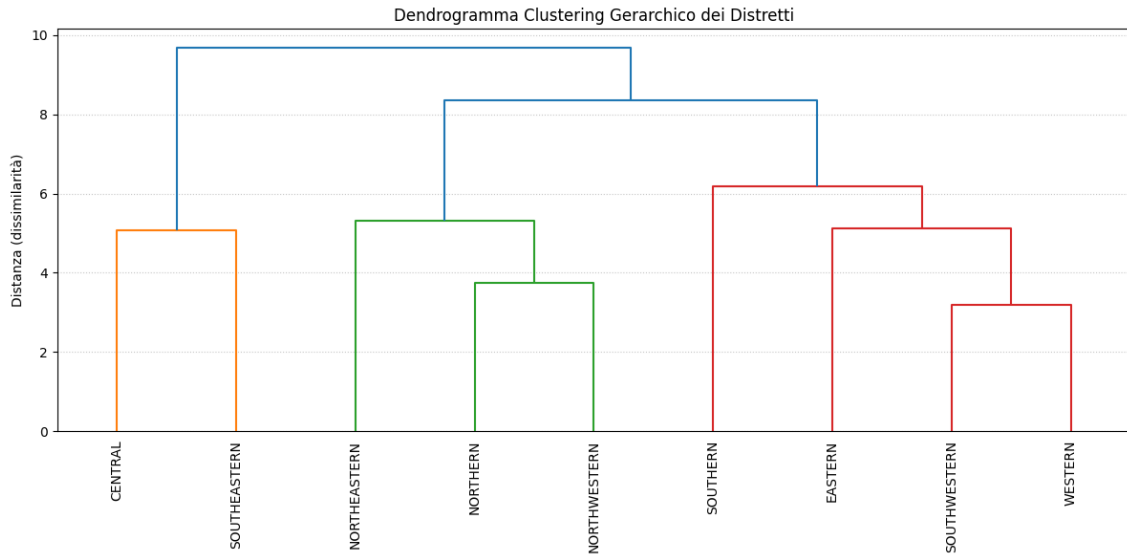


Figura 11: Dendrogramma

Come si può osservare dalla Figura 11, i risultati di K-Means sono confermati, aggiungendo un ulteriore livello di dettaglio, mostrando **come e quanto** i distretti sono simili tra loro.

1. **Conferma del Cluster 3:** i distretti *SOUTHWESTERN* e *WESTERN* sono i primi in assoluto a fondersi (la linea rossa orizzontale che li unisce è la più bassa, con distanza 3.2). Ciò significa che sono la coppia di distretti con i profili di criminalità **più simili in assoluto** in tutta l'analisi. Questo conferma che il nucleo del K-Means Cluster 3 è un raggruppamento molto forte.
2. **Conferma del Cluster 0:** i distretti *NORTHERN* e *NORTHWESTERN* sono uniti dalla linea verde ad una distanza di 3.8 ed è la seconda fusione più stretta, confermando che anch'essi sono un raggruppamento **molto logico e robusto**.
3. **Gli altri raggruppamenti:** per il Cluster 1, *CENTRAL* e *SOUTHEASTERN* (linea arancione) si fondono a una distanza maggiore (5.1), *EASTERN* si unisce al gruppo "rosso" (Cluster 3) a distanza 5.1, mentre *NORTHEASTERN* si unisce al gruppo "verde" (Cluster 0) a distanza 5.3.
4. **Il "Ponte" (Cluster 2):** come nell'analisi K-Means, *NORTHEASTERN* e *SOUTHERN* **non si uniscono direttamente tra loro**. Il dendrogramma suggerisce che *NORTHEASTERN* ha più affinità con il Cluster 0 e *SOUTHERN* ha più affinità con il Cluster 3. Questo rafforza l'idea che il K-Means Cluster 2 sia il raggruppamento meno compatto, composto da distretti "ponte".



In conclusione, il dendrogramma arricchisce il K-Means, esplicitando come i Cluster 0,1 e 3 sono raggruppamenti **molto naturali e robusti**. Al contrario, il Cluster 2 è il raggruppamento **più debole** ed ambiguo, composto da distretti ponte che hanno affinità anche con altri gruppi.

### 3.7 Visualizzazione geografica dei cluster

La prossima fase integra i risultati del clustering con i dati geografici dei distretti di polizia, consentendo una lettura spaziale dei gruppi individuati.

Utilizzando la libreria Folium, viene generata una mappa interattiva che:

- Mostra i distretti colorati in base al cluster di appartenenza;
- Evidenzia la coerenza spaziale dei gruppi (ad esempio distretti contigui nello stesso cluster);
- Consente di verificare se esistono pattern territoriali coerenti con i risultati statistici.

Le coordinate spaziale e confini territoriali dei distretti sono stati scaricati direttamente dal sito Open Baltimore.

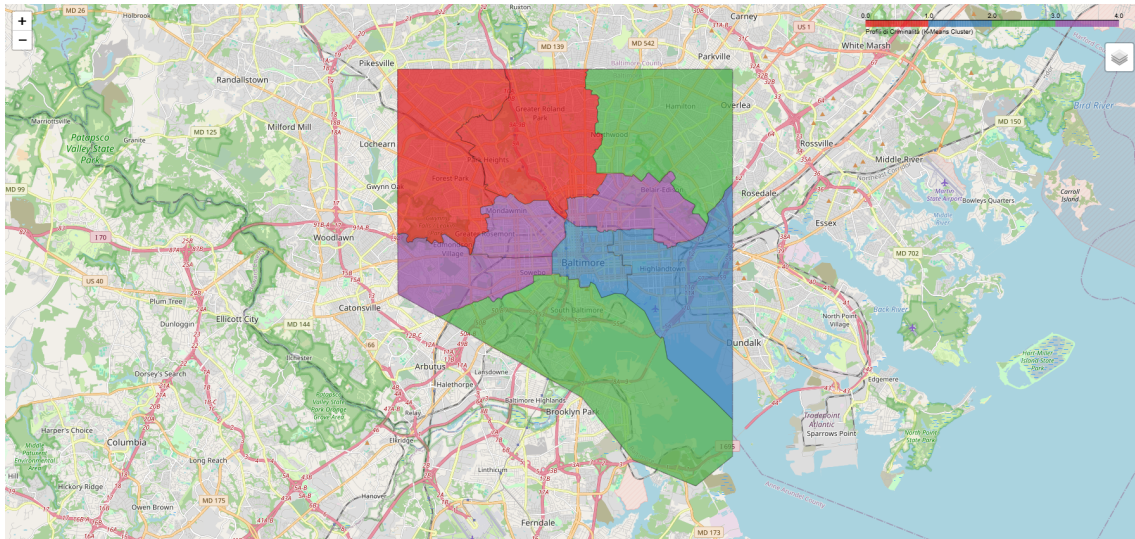


Figura 12: Visualizzazione mappa dei Cluster dei distretti

La mappa (Figura 12) mostra come i distretti sono raggruppati geograficamente nei 4 Cluster:

1. **Cluster 0 (Rosso - Rapine con armi da fuoco):** comprende i distretti *NORTHERN* e *NORTHWESTERN*, formando un blocco contiguo a **Nord-Ovest**.

2. **Cluster 1 (Blu - Crimini Notturmi/Esterni)**: comprende *CENTRAL* e *SOUTHEASTERN* e copre l'area centrale e sud-orientale della città (le aree del centro e del porto).
3. **Cluster 2 (Verde - Rapine a Proprietà/Interne)**: comprende *NORTHEASTERN* e *SOUTHERN* ed è il cluster più "sparso", con un distretto a **Nord-Est** e uno a **Sud**. La mappa conferma che è il cluster più "debole", mostrando che i suoi membri non sono geograficamente uniti.
4. **Cluster 3 (Viola - Aggressioni Diurne)**: comprende *EASTERN*, *WESTERN* e *SOUTHWESTERN*, formando una "cintura" o "arco" coeso che avvolge il centro della città. Questa è un'area geografica chiaramente definita con un profilo distintivo di "aggressioni diurne".

### 3.8 Clustering con DBSCAN per l'identificazione di Hotspots

A differenza del K-Means, che raggruppa tutti i distretti in un numero predefinito di cluster, l'algoritmo **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** viene utilizzato per un obiettivo diverso: **identificare "focolai" (hotspots) di criminalità** basati sulla loro diversità, indipendentemente dai confini amministrativi dei distretti.

Utilizza le coordinate (*Latitude* e *Longitude*) che vengono **standardizzate** attraverso **StandardScaler**, per assicurare che le coordinate abbiano lo stesso peso nell'analisi, poiché DBSCAN si basa sulla distanza Euclidea.

DBSCAN richiede due parametri:

- *MIN\_SAMPLES*: numero minimo di punti per formare un hotspot, che è fissato a 50.
- *EPSILON*: raggio di ricerca attorno ad un punto; per trovarlo, è stato generato un **Grafico K-Distance** (Figura 13), che ordina tutti i punti in base alla loro distanza dal 50° vicino più prossimo.

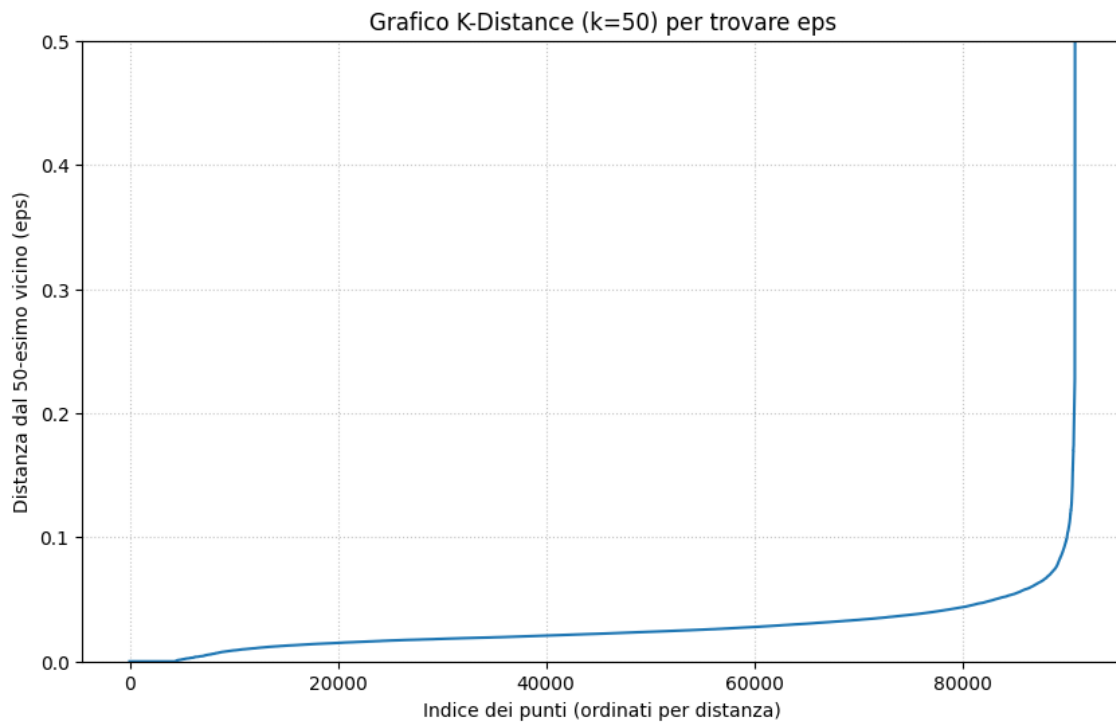


Figura 13: Grafico K-Distance per Epsilon

Come mostra il grafico zoomato, la curva rimane quasi piatta fino a circa l'indice 85.000, per poi impennarsi bruscamente. Il punto in cui inizia questa impennata (il gomito) si trova a un valore di distanza di circa *0.06*, che è stato scelto come valore di *EPSILON*.

```

1 # Definizione dei parametri DBSCAN
2 EPSILON = 0.06
3 MIN_SAMPLES = 50
4
5 # Esecuzione di DBSCAN
6 db = DBSCAN(eps=EPSILON, min_samples=MIN_SAMPLES, metric='euclidean')
7 cluster_db = db.fit_predict(coords_scaled)
8
9 # Aggiunta dei risultati di DBSCAN al DataFrame originale
10 df['hotspot_cluster'] = cluster_db
11
12 print("Punti per cluster (hotspot): (-1 = Rumore/Non-Hotspot)")
13 print(df['hotspot_cluster'].value_counts())

```

```

1 Punti per cluster (hotspot): (-1 = Rumore/Non-Hotspot)
2 hotspot_cluster
3 0 81126

```

```

4  1 2937
5  3 2652
6 -1 1384
7  5 996
8  2 472
9 13 218
10 11 178
11 6 121
12 7 109
13 4 105
14 16 92
15 14 91
16 12 86
17 10 82
18 8 66
19 15 65
20 9 59
21 17 26
22 Name: count, dtype: int64

```

Listing 6: Hotspots dei cluster

L'output ha raggruppato i crimini in 19 cluster, che possono essere distinti in:

- **Rumore (Cluster -1):** l'algoritmo ha identificato *1.384* crimini (circa l'1.5% del totale) come rumore. Questi sono incidenti isolati, troppo lontani da altri crimini per essere considerati parte di un focolaio.
- **Super-Cluster (Cluster 0):** un massiccio cluster da *81.126* punti. Questo non è un singolo hotspot, ma rappresenta l'area urbana principale di Baltimora, dove la densità di crimini è generalmente alta ovunque. Possiamo considerarlo come il "rumore di fondo" della vita cittadina.
- **Veri Hotspot (Cluster 1,3,5,2,etc.):** la scoperta più importante è l'identificazione di **17 cluster distinti** (da 0 a 17, escluso il Super-Cluster 0). Questi sono veri e propri focolai di criminalità, aree dove la densità di crimini è significativamente superiore rispetto al resto della città. L'analisi ha isolato:
  - Due hotspot principali molto grandi (**Cluster 1 - 2.937** e **Cluster 3 - 2.652**)
  - Una serie di **14 hotspot** più piccoli ma comunque abbastanza densi.

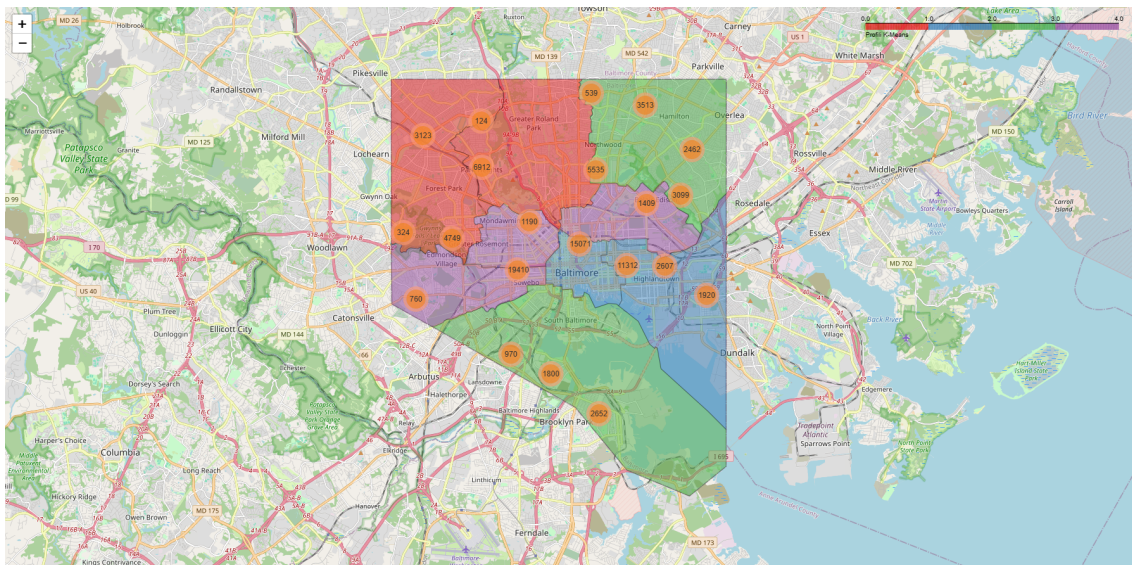


Figura 14: Hotspots nei vari cluster

Il risultato della mappa (Figura 14) conferma i risultati:

- **Cluster 3 (Viola - Aggressioni diurne):** i distretti *EASTERN*, *WESTERN* e *SOUTHWESTERN* contengono gli hotspot **più densi in assoluto** (i grappoli da 19.410 e 15.071, quest'ultimo conteso con il Cluster 1). Questo conferma che questo "profilo" statistico corrisponde all'area geografica con la più alta concentrazione di crimini.
- **Cluster 0 (Rosso - Rapine con armi da fuoco):** i distretti *NORTHERN* e *NORTHWESTERN* contengono anch'essi hotspot estremamente densi (6.192, 4.749 e 3.123)
- **Cluster 1 (Blu - Crimini notturni/esterni):** i distretti *CENTRAL* e *SOUTHEASTERN* hanno hotspot significativi (15.071, 11.312 e 2.607), specialmente nell'area centrale.
- **Cluster 2 (Verde - Rapine a proprietà):** questo cluster ha hotspot più piccoli e sparsi (5.535 conteso con il cluster 0, 2.462, 3.099 conteso con il cluster 3, 2.652, 1.800), confermando che è un profilo di criminalità meno "concentrato".

### 3.9 Conclusioni generali

L'analisi di clustering sui distretti di polizia ha permesso di ricavare una rappresentazione strutturata e informativa dei principali pattern di criminalità presenti sul

territorio. Attraverso un processo rigoroso di preparazione dei dati, standardizzazione delle variabili e costruzione di profili percentuali, è stato possibile applicare l'algoritmo K-Means in modo efficace e interpretabile.

L'uso combinato dei metodi Elbow e Silhouette ha indicato come ottimale una suddivisione in quattro cluster, ognuno caratterizzato da specifiche combinazioni di tipologie di reato, armi utilizzate, fasce orarie e contesti Inside/Outside. La successiva visualizzazione tramite PCA ha confermato la buona separazione tra i gruppi, evidenziando due assi interpretativi principali:

- PC1, associato al livello di gravità del crimine e alla presenza di armi da fuoco rispetto a reati meno violenti;
- PC2, legato al momento della giornata e alla distinzione tra aggressioni e rapine.

L'analisi dei centroidi e la costruzione della heatmap hanno mostrato che alcune caratteristiche come *COMMON ASSAULT*, *Weapon\_HANDS* e il rapporto *Inside/Outside* sono frequentemente presenti in tutta la città, mentre altre risultano determinanti per distinguere i cluster. Il calcolo del Lift ha fornito una prospettiva quantitativa più precisa, evidenziando quali caratteristiche risultano significativamente sopra o sotto la media nei diversi gruppi.

Nel complesso, l'insieme delle analisi ha permesso di delineare profili criminologici distinti, tra cui:

- cluster con maggiore incidenza di **rapine e uso di armi da fuoco**;
- cluster caratterizzati da **reati notturni all'esterno**;
- cluster dominati da **aggressioni, soprattutto durante il giorno**;
- cluster con profili più equilibrati e privi di elementi particolarmente dominanti.

Queste informazioni offrono una base solida per interpretazioni operative e strategiche, come la pianificazione delle risorse, l'allocazione delle pattuglie e l'identificazione di aree con specifiche criticità.

## 4 Timeseries

L'obiettivo dell'analisi è quello di esplorare, pulire e comprendere l'andamento temporale e spaziale dei crimini, applicando tecniche di **analisi di serie temporali e forecasting**, per prevedere l'evoluzione dei fenomeni criminali in determinati distretti o periodi.

Per l'analisi verranno utilizzate librerie **Python** come **pandas**, **matplotlib**, **seaborn**, **statsmodels**, **numpy**, **pmdarima** e **sklearn**, che consentiranno di effettuare una **pulizia accurata dei dati**, un' **analisi esplorativa** e la **modellazione del modello di clustering**.

### 4.1 Preparazione dati

Il processo ETL per l'analisi di forecasting viene approfondito attraverso una **aggregazione** per periodo per studiare l'evoluzione dei crimini nel tempo. Questa operazione consente di generare una serie temporale settimanale.

```
1 crime_time_series = (  
2 crime_data  
3 .set_index('DateTime')  
4 .resample('W')['Total Incidents']  
5 .sum()  
6 .reset_index())
```

Il risultato finale ottenuto tramite le operazioni di pulizia preliminari, è il seguente.

```
1 --- Valori mancanti dopo il fill ---  
2 Location 0  
3 Inside/Outside 0  
4 Weapon 0  
5 District 0  
6 Neighborhood 0  
7 dtype: int64  
8  
9 === Dimensione dataset pulito ===  
10 Serie temporale settimanale: (307, 2)  
11  
12 === Esempio serie temporale settimanale ===  
13 DateTime Total Incidents  
14 2011-01-02 281  
15 2011-01-09 800  
16 2011-01-16 750
```

```

17 2011-01-23 804
18 2011-01-30 667
19
20 === Statistiche descrittive della serie temporale settimanale ===
21         DateTime Total Incidents
22 count 307 307.000000
23 mean 2013-12-08 00:00:00 917.247557
24 min 2011-01-02 00:00:00 281.000000
25 25% 2012-06-20 12:00:00 851.000000
26 50% 2013-12-08 00:00:00 932.000000
27 75% 2015-05-27 12:00:00 1003.000000
28 max 2016-11-13 00:00:00 1114.000000
29 std NaN 116.174862

```

Listing 7: Output del processo di pulizia e aggregazione settimanale

## 4.2 Analisi esplorativa del dataset

Si effettuerà una prima analisi visiva del dataset graficando l'andamento della serie temporale dei crimini a Baltimora, in modo tale da poter avanzare una prima ipotesi sulla stazionarietà della serie.

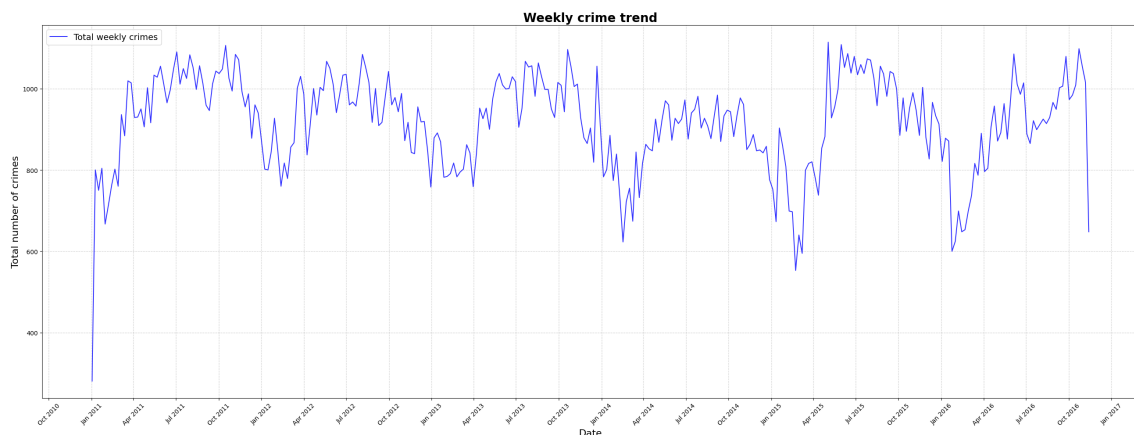


Figura 15: Weekly Crime Trend

Osservando il grafico (Figura 15), si possono notare alcune caratteristiche che suggeriscono che la serie **non è stazionaria**:

1. **Media non costante nel tempo**: ci sono periodi in cui il numero totale di incidenti settimanali è mediamente più alto (ad esempio tra il 2011 e il 2013, di nuovo intorno al 2015-2016). Altri periodi mostrano una diminuzione visibile (ad esempio inizio 2015 e inizio 2016). Questo indica che la **media varia nel tempo**.



2. **Varianza non costante nel tempo:** l'ampiezza delle fluttuazioni ("dispersione" dei valori) non è omogenea: in certi intervalli la variabilità sembra più elevata, mentre in altri le oscillazioni sono più contenute. Anche la **varianza**, quindi, sembra **cambiare nel tempo**.
3. **Possibile stagionalità:** il grafico mostra ciclicità annuale o semestrale: periodi ricorrenti in cui i crimini aumentano o diminuiscono, compatibili con **pattern stagionali** (ad esempio aumenti estivi o cali invernali). Questo tipo di oscillazione periodica è tipico di una serie **non stazionaria**, con componente stagionale.
4. **Valori più contenuti in certi intervalli:** si notano cali marcati in corrispondenza di alcuni anni (ad esempio inizio 2015 e 2016), forse legati a fattori specifici (politiche di sicurezza, eventi eccezionali, cambiamenti nei dati).

In sintesi, la serie sembra **non essere stazionaria**, poiché **media e varianza variano nel tempo** e sono presenti **elementi di stagionalità**. Per analisi predittive è necessario **stazionarla** tramite differenziazione o rimozione della stagionalità. Per effettuare un'analisi più approfondita, si decomporrà la serie nelle sue componenti stagionali, di trend e residuali.

```
1 decomposition = seasonal_decompose(  
2     crime_time_series.set_index('DateTime')['Total Incidents'],  
3     model='additive',  
4     period=52  
5 )
```

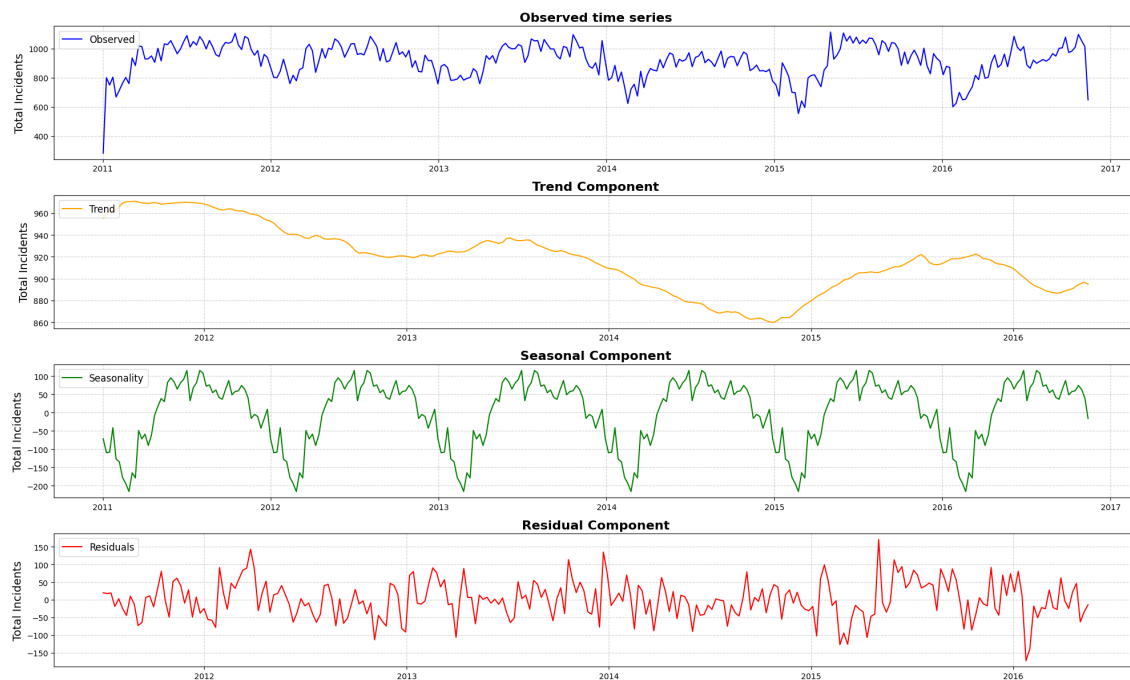


Figura 16: Decomposizione della serie

Analizzando il grafico della decomposizione della serie temporale (Figura 16), si osservano:

1. **Componente di Trend:** il trend mostra **una diminuzione graduale dal 2011 al 2014**, seguita da **una lieve risalita tra il 2015 e il 2016**. Questo suggerisce che, nel lungo periodo, il **numero medio di crimini settimanali tende a calare leggermente**, ma con una ripresa negli ultimi anni del campione. Quindi, **la media non è costante nel tempo**, confermando che la serie non è stazionaria in livello.
2. **Componente stagionale:** si osserva una **chiara periodicità annuale**, con oscillazioni regolari e simmetriche intorno allo zero. La stagionalità è **molto marcata e stabile nel tempo**, indicando che i crimini tendono ad aumentare e diminuire in determinati periodi dell'anno (ad esempio più incidenti nei mesi estivi, meno in quelli invernali). Questa regolarità suggerisce una **stagionalità fissa e ripetitiva**, il che è tipico di fenomeni sociali o ambienti influenzati da condizioni ricorrenti.
3. **Componente residuale:** i residui oscillano intorno allo zero senza una struttura evidente, ma **non appaiono completamente bianchi**: in certi periodi (ad esempio 2015) sembrano più dispersi o con outlier. Ciò indica che **parte della varianza non è spiegata** né dal trend né dalla stagionalità, ma non

emergono pattern sistematici forti. La varianza dei residui appare **piuttosto costante nel tempo**, un buon segno per modellazione successiva.

In sintesi, la serie **non è stazionaria in livello**, perché presenta **trend e stagionalità**. Dopo la rimozione del trend e della stagionalità o una differenziazione stagionale, i **residui potrebbero risultare stazionari** e quindi adatti a modelli successivi. Per confermare l'ipotesi, si effettua l'**ADF Test (Augmented Dickey-Fuller)** e si misura **p-value**, che per serie stazionarie dovrebbe essere inferiore alla soglia di *0.05*.

```
1 # Test di stazionariet ADF
2 adf_test = adfuller(crime_time_series['Total Incidents'])
3
4 adf_results = pd.Series({"ADF Test Statistic": adf_test[0],
5                          "p-value": adf_test[1],
6                          "Used Lag": adf_test[2],
7                          "Number of Observations": adf_test[3],
8                          "Critical Value (1%)": adf_test[4]['1%'],
9                          "Critical Value (5%)": adf_test[4]['5%'],
10                         "Critical Value (10%)": adf_test[4]['10%']})
11
12 print("\n=== Risultati del test di stazionariet ADF ===")
13 adf_results
```

```
1 ADF Test Statistic -4.303567
2 p-value 0.000438
3 Used Lag 9.000000
4 Number of Observations 297.000000
5 Critical Value (1%) -3.452561
6 Critical Value (5%) -2.871321
7 Critical Value (10%) -2.571982
8 dtype: float64
```

Listing 8: Risultati test ADF

In contrapposizione all'analisi effettuata, osservando i valori dell'ADF Test si può **rifiutare l'ipotesi nulla (H0)** con elevata confidenza (livello superiore al 99%) e concludere che la serie è **stazionaria**. Tuttavia, ci sono alcune considerazioni da tenere a mente:

- **Statisticamente**, l'ADF dice che la serie può essere trattata come stazionaria.

- **Visivamente**, però, rimangono segnali di trend e stagionalità, quindi per l'utilizzo di modelli come ARIMA/SARIMA è comunque consigliabile una **differenziazione (o rimozione della stagionalità)** per sicurezza.

Quindi, si procederà con due approcci di differenziazione:

- **Semplice**: per rimuovere il trend, con l'effetto di trasformare la serie che oscilla attorno a una media costante, di fatto "appiattendolo" il trend.
- **Stagionale**: per rimuovere pattern annuali o periodici, eliminando la dipendenza ciclica annuale; confrontando ogni settimana con la stessa settimana dell'anno prima, si annulla l'effetto sistematico legato a quel particolare periodo dell'anno.

```
1 # Differenziazione semplice
2 crime_diff = crime_time_series['Total Incidents'].diff().dropna()
3
4 # Differenziazione stagionale (period=52 settimane)
5 crime_seasonal_diff = crime_time_series['Total Incidents'].diff(52).
   dropna()
```

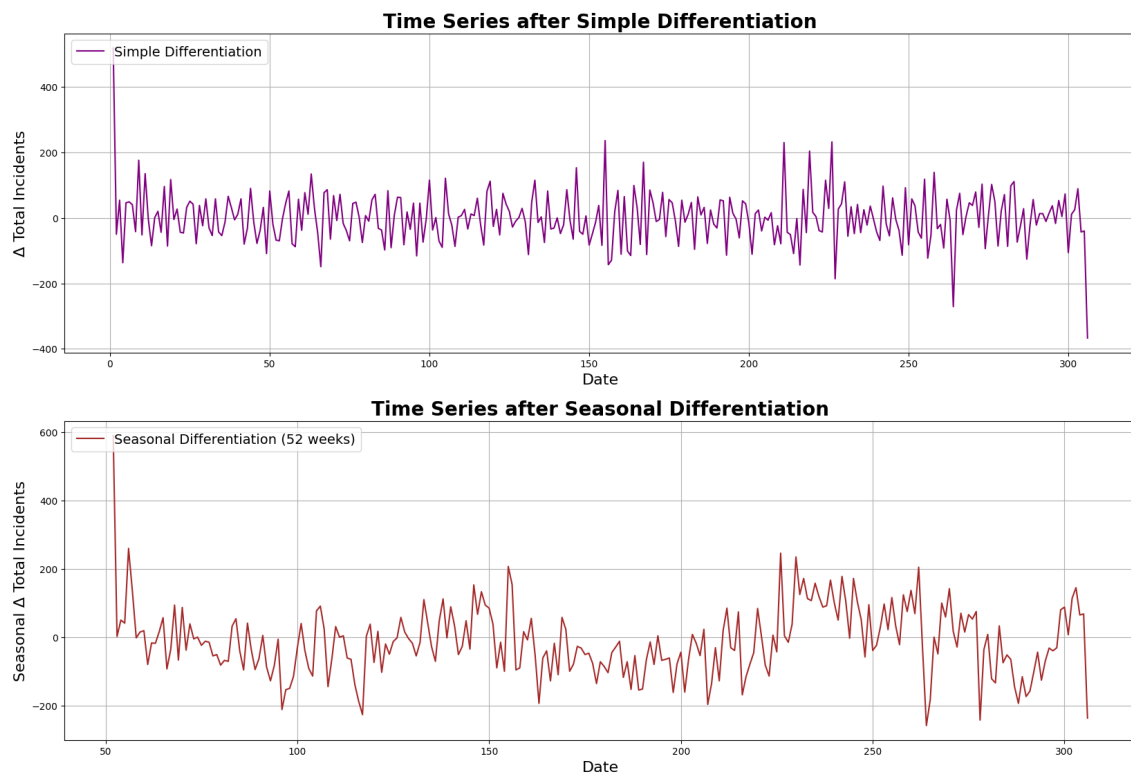


Figura 17: Differenziazioni semplice e stagionale

```

1  === Risultati del test di stazionariet ADF sulla serie differenziata semplice
    ↪ ===
2  ADF Test Statistic -7.476872e+00
3  p-value 4.887433e-11
4  Used Lag 5.000000e+00
5  Number of Observations 3.000000e+02
6  Critical Value (1%) -3.452337e+00
7  Critical Value (5%) -2.871223e+00
8  Critical Value (10%) -2.571929e+00
9  dtype: float64
10
11 === Risultati del test di stazionariet ADF sulla serie differenziata
    ↪ stagionale ===
12 ADF Test Statistic -4.166292
13 p-value 0.000751
14 Used Lag 5.000000
15 Number of Observations 249.000000
16 Critical Value (1%) -3.456888
17 Critical Value (5%) -2.873219
18 Critical Value (10%) -2.572994
19 dtype: float64

```

Listing 9: Risultati test ADF per le differenziazioni

Osservando i grafici (Figura 17) e i risultati dei test ADF (Listato 9), si può concludere che:

1. **Differenziazione semplice:** la statistica ADF (-7.48) è **molto più piccola** del valore critico al 1% (-3.45). Il p-value è **praticamente zero**. Conclusione: la serie differenziata semplice è **fortemente stazionaria**, il trend è stato completamente rimosso e ora la media e la varianza sono costanti nel tempo.
2. **Differenziazione stagionale (lag = 52 settimane):** anche qui la statistica ADF è **inferiore** al valore critico al 1% (-3.46). Il p-value è ancora **molto basso** (<0.01). Conclusione: la serie differenziata stagionale è anch'essa **stazionaria**, ma l'effetto è **meno marcato** rispetto alla differenziazione semplice.

In sintesi, la **differenziazione semplice** ha reso la serie fortemente stazionaria eliminando il trend principale, mentre la **differenziazione stagionale** ha rimosso le oscillazioni annuali, ma non riduce tanto la dipendenza come prima. Entrambe le serie, quindi, risultano **statisticamente stazionarie**, ma per motivi diversi.

Per modelli come **SARIMA**, dove è utile catturare sia trend che stagionalità, la soluzione migliore è **applicare entrambe le differenziazioni**, verificando di nuovo la stazionarietà e la stabilità visiva, dato che, in genere, la doppia differenziazione fornisce una serie finale stazionaria in media e varianza, pronta per il fitting di un modello ARIMA/SARIMA.

```
1 # Differenziazione completa (semplice + stagionale)
2 crime_diff_full = crime_time_series['Total Incidents'].diff().diff(52)
   .dropna()
```

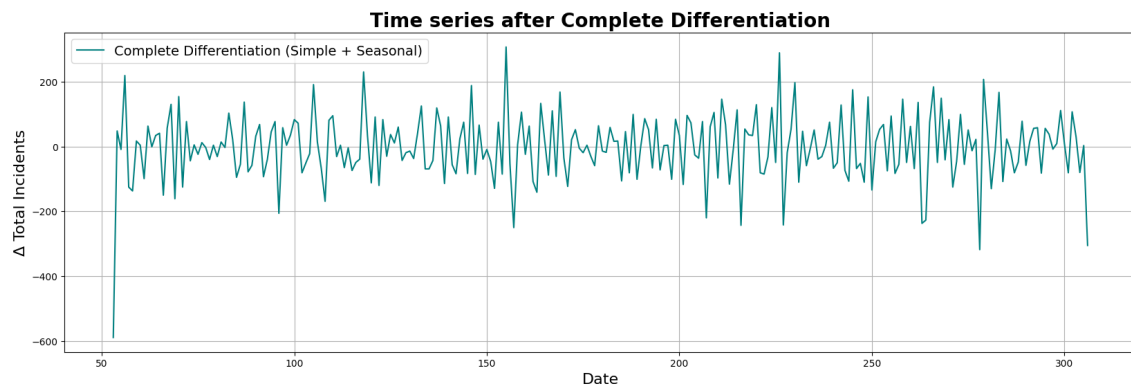


Figura 18: Differenziazione totale

Nel grafico (Figura 18) possiamo osservare che la serie differenziata completa (semplice + stagionale) oscilla chiaramente **intorno allo zero**, senza trend a lungo termine evidente. Inoltre, la varianza è **piuttosto costante**, con un'ampiezza delle oscillazioni relativamente consistenti lungo tutto l'asso temporale. Infine, non si osservano più pattern stagionali regolari.

```
1 === Risultati del test di stazionariet ADF sulla serie completamente
   ↳ differenziata ===
2 ADF Test Statistic -1.157592e+01
3 p-value 3.022940e-21
4 Used Lag 4.000000e+00
5 Number of Observations 2.490000e+02
6 Critical Value (1%) -3.456888e+00
7 Critical Value (5%) -2.873219e+00
8 Critical Value (10%) -2.572994e+00
9 dtype: float64
```

Listing 10: Risultati test ADF per differenziazione totale

Osservando i risultati del test di stazionarietà ADF applicato alla serie differenziata completa (Listato 10), si può concludere che la serie è **pienamente stazionaria**.

### 4.3 Analisi dei correlogrammi e delle componenti

Si può quindi procedere con l'analisi dei **correlogrammi ACF e PACF** per stimare i parametri ( $p$ ,  $d$ ,  $q$ ) del modello SARIMA.

```
1 # Grafico ACF
2 plot_acf(crime_diff_full, lags=52, ax=axes[0])
3 axes[0].set_title('Autocorrelation Function (ACF)', fontsize=16,
4                   fontweight='bold')
5 axes[0].set_xlabel('Lags', fontsize=14)
6 axes[0].set_ylabel('Autocorrelation', fontsize=14)
7 axes[0].grid(True, linestyle='--', alpha=0.6)
8
9 # Grafico PACF
10 plot_pacf(crime_diff_full, lags=52, ax=axes[1])
11 axes[1].set_title('Partial Autocorrelation Function (PACF)', fontsize
12                  =16, fontweight='bold')
13 axes[1].set_xlabel('Lags', fontsize=14)
14 axes[1].set_ylabel('Partial Autocorrelation', fontsize=14)
15 axes[1].grid(True, linestyle='--', alpha=0.6)
```

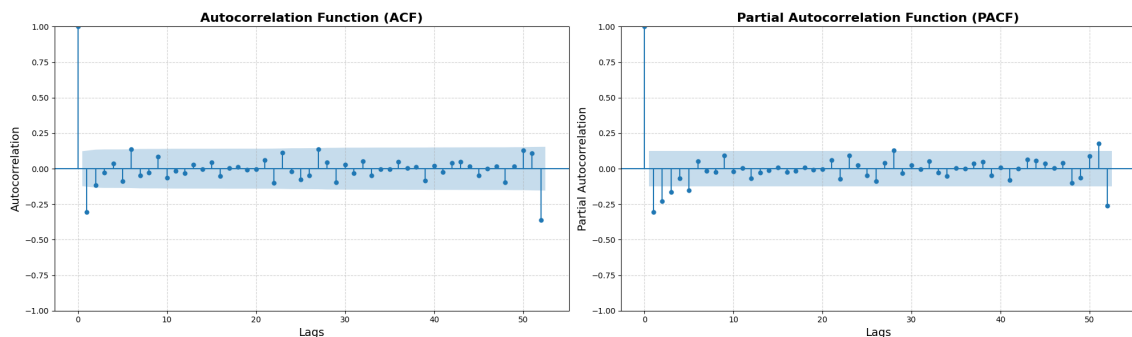


Figura 19: Autocorrelogrammi

Per identificare i parametri **p** (ordine AR) e **q** (ordine MA), si osservano i primi lag dei grafici:

- **Grafico ACF:** c'è un picco negativo statisticamente significativo al **lag 1**. Subito dopo, i valori successivi cadono rapidamente all'interno della banda di confidenza (area azzurra). Questo pattern con un taglio netto e improvviso dopo il primo lag, è caratteristico di un **modello a Media Mobile (MA)**. L'ordine del modello è dato dall'ultimo lag significativo prima del taglio, suggerendo quindi un parametro **q = 1**.

- **Grafico PACF:** anche qui c'è un picco negativo significativo al **lag 1**, e i valori successivi tendono a cadere rapidamente all'interno della banda di confidenza. Un taglio netto nel PACF è tipico di un **modello Autoregressivo (AR)**, suggerendo un parametro  $p = 1$ .

Avendo indicazioni sia per un modello AR(1) sia per un MA(1) e analizzando entrambi i grafici, si osserva che il grafico ACF è molto netto e pulito, rendendo l'ipotesi di un **modello MA(1)** ( $q = 1, p = 0$ ) leggermente più probabile e un ottimo punto di partenza.

Per determinare i parametri stagionali **P** e **Q**, si osservano i lag specificamente vicini al 52:

- **Grafico ACF:** si nota un picco negativo statisticamente significativo al **lag 52**. Questo picco è isolato, il che è un segnale molto forte per un modello di **Media Mobile Stagionale (SMA)**. Questo conferma la precedente ipotesi, suggerendo un valore di  $Q = 1$ .
- **Grafico PACF:** analogamente, si osserva un picco negativo significativo al **lag 52**. Anche questo picco sembra essere isolato, suggerendo un modello **Autoregressivo Stagionale (SAR)** e un valore di  $P = 1$ .

Verrà eseguito **Auto Arima**, specificando il periodo stagionale e forzando la differenziazione per trovare il modello migliore.

```

1 sarima_model = pm.auto_arima(
2     crime_time_series['Total Incidents'], # La serie temporale
3     seasonal=True, # Abilita la ricerca di un modello stagionale
4     m=52, # Il periodo a 52 settimane
5
6     # Parametri per guidare la ricerca
7     d=1, # Forza una differenziazione semplice (per il trend)
8     D=1, # Forza una differenziazione stagionale
9     start_p=0, start_q=0, # Punto di partenza per i parametri non
      stagionali
10    start_P=0, start_Q=0, # Punto di partenza per i parametri
      stagionali
11    max_p=2, max_q=2, # Limite massimo per p e q
12    max_P=2, max_Q=2, # Limite massimo per P e Q
13
14    # Parametri per il controllo del processo
15    trace=True, # Mostra i modelli che sta testando

```



```

16     error_action='ignore', # Ignora i modelli che non convergono
17     suppress_warnings=True, # Sopprime i warning
18     stepwise=True # Usa un approccio efficiente per la ricerca
19 )
20
21 # Stampa il sommario del modello migliore trovato
22 print(sarima_model.summary())

```

```

1 Performing stepwise search to minimize aic
2 ARIMA(0,1,0)(0,1,0)[52] : AIC=3084.654, Time=0.57 sec
3 ARIMA(1,1,0)(1,1,0)[52] : AIC=3001.796, Time=16.69 sec
4 ARIMA(0,1,1)(0,1,1)[52] : AIC=inf, Time=57.68 sec
5 ARIMA(1,1,0)(0,1,0)[52] : AIC=3057.527, Time=1.07 sec
6 ARIMA(1,1,0)(2,1,0)[52] : AIC=2989.641, Time=60.63 sec
7 ARIMA(1,1,0)(2,1,1)[52] : AIC=2991.259, Time=176.38 sec
8 ARIMA(1,1,0)(1,1,1)[52] : AIC=2990.250, Time=118.76 sec
9 ARIMA(0,1,0)(2,1,0)[52] : AIC=inf, Time=57.87 sec
10 ARIMA(2,1,0)(2,1,0)[52] : AIC=2974.791, Time=87.72 sec
11 ARIMA(2,1,0)(1,1,0)[52] : AIC=2988.008, Time=22.12 sec
12 ARIMA(2,1,0)(2,1,1)[52] : AIC=2976.337, Time=208.68 sec
13 ARIMA(2,1,0)(1,1,1)[52] : AIC=2975.967, Time=142.43 sec
14 ARIMA(2,1,1)(2,1,0)[52] : AIC=2960.771, Time=156.23 sec
15 ARIMA(2,1,1)(1,1,0)[52] : AIC=2966.596, Time=42.47 sec
16 ARIMA(2,1,1)(2,1,1)[52] : AIC=2962.539, Time=361.75 sec
17 ARIMA(2,1,1)(1,1,1)[52] : AIC=2961.167, Time=187.17 sec
18 ARIMA(1,1,1)(2,1,0)[52] : AIC=2958.777, Time=130.16 sec
19 ARIMA(1,1,1)(1,1,0)[52] : AIC=2964.613, Time=40.86 sec
20 ARIMA(1,1,1)(2,1,1)[52] : AIC=2960.546, Time=263.86 sec
21 ARIMA(1,1,1)(1,1,1)[52] : AIC=2959.168, Time=144.58 sec
22 ARIMA(0,1,1)(2,1,0)[52] : AIC=2960.797, Time=86.79 sec
23 ARIMA(1,1,2)(2,1,0)[52] : AIC=inf, Time=377.63 sec
24 ARIMA(0,1,2)(2,1,0)[52] : AIC=2958.837, Time=97.30 sec
25 ARIMA(2,1,2)(2,1,0)[52] : AIC=inf, Time=324.28 sec
26 ARIMA(1,1,1)(2,1,0)[52] intercept : AIC=2960.279, Time=146.04 sec
27
28 Best model: ARIMA(1,1,1)(2,1,0)[52]
29 Total fit time: 3309.859 seconds
30
31 SARIMAX Results
=====
31
32 Dep. Variable: y No. Observations: 307
33 Model: SARIMAX(1, 1, 1)x(2, 1, [], 52) Log Likelihood -1474.389
34 Date: Fri, 14 Nov 2025 AIC 2958.777
35 Time: 16:34:31 BIC 2976.464

```

```

36 Sample: 0 HQIC 2965.892
37                                     - 307
38 Covariance Type: opg
39 =====
40      ↪
41      coef std err z P>|z| [0.025 0.975]
42 -----
43      ↪
44 ar.L1 0.2262 0.079 2.854 0.004 0.071 0.382
45 ma.L1 -0.8219 0.051 -16.055 0.000 -0.922 -0.722
46 ar.S.L52 -0.6176 0.067 -9.163 0.000 -0.750 -0.485
47 ar.S.L104 -0.2403 0.064 -3.780 0.000 -0.365 -0.116
48 sigma2 5925.4936 387.760 15.281 0.000 5165.499 6685.488
49 =====
50      ↪
51 Ljung-Box (L1) (Q): 1.32 Jarque-Bera (JB): 199.54
52 Prob(Q): 0.25 Prob(JB): 0.00
53 Heteroskedasticity (H): 1.07 Skew: -0.66
54 Prob(H) (two-sided): 0.76 Kurtosis: 7.14
55 =====
56      ↪

```

Listing 11: Risultati Auto Arima

Come si può osservare dal Listato 11, il modello migliore trovato è quello con parametri  $p=1$ ,  $d=1$ ,  $q=1$ ,  $P=2$ ,  $D=1$ ,  $Q=0$ .

- **Componente non stagionale:**

- **$d=1$ :** conferma che è stata necessaria una differenziazione semplice per rimuovere il trend.
- **$p=1$ :** il numero di incidenti di una settimana è influenzato dal numero di incidenti della settimana precedente.
- **$q=1$ :** il modello viene corretto anche in base all'errore di previsione commesso nella settimana precedente.

- **Componente stagionale:**

- **$D=1$ :** conferma che sia stata necessaria una differenziazione stagionale per rimuovere il ciclo annuale.
- **$P=2$ :** il numero di incidenti di una settimana è influenzato non solo dal valore della stessa settimana dell'anno precedente (lag 52), ma anche da quello della stessa settimana di due anni prima (lag 104). Questo indica un pattern stagionale molto forte e persistente nel tempo.

- **Q=0**: il modello non ha trovato necessario includere un termine di media mobile stagionale.

- **Diagnostica dei residui:**

- **Ljung-Box ( $\text{Prob}(Q) = 0.25$ )**: il p-value è maggiore di 0.05, indicando che i residui non sono autocorrelati. Il modello ha catturato con successo tutta la struttura temporale presente nei dati, lasciando solo "rumore bianco".
- **Heteroskedasticity ( $\text{Prob}(H) = 0.76$ )**: il p-value è maggiore di 0.05, confermando che la varianza degli errori è costante nel tempo (omoschedasticità). Il modello è stabile e non ha periodi in cui commette errori sistematicamente più grandi o più piccoli.
- **Jarque-Bera ( $\text{Prob}(JB) = 0.00$ )**: il p-value è minore di 0.05, mostrando che i residui non seguono una distribuzione normale. La **Kurtosis** alta (7.14) suggerisce che la distribuzione degli errori ha "code più pesanti", ovvero ci sono più valori estremi (outlier) di quanto ci si aspetterebbe. Sebbene non sia ideale, questo è molto comune nelle serie storiche reali e, per scopi di previsione, è meno grave della presenza di autocorrelazione.

Si può procedere con la diagnostica e l'analisi dei residui prodotti da tale modello, per assicurarci che non ci siano pattern particolari, cercando media e varianza costante.

```
1 # Diagnostica del modello SARIMA
2 sarima_model.plot_diagnostics(figsize=(20, 12));
```

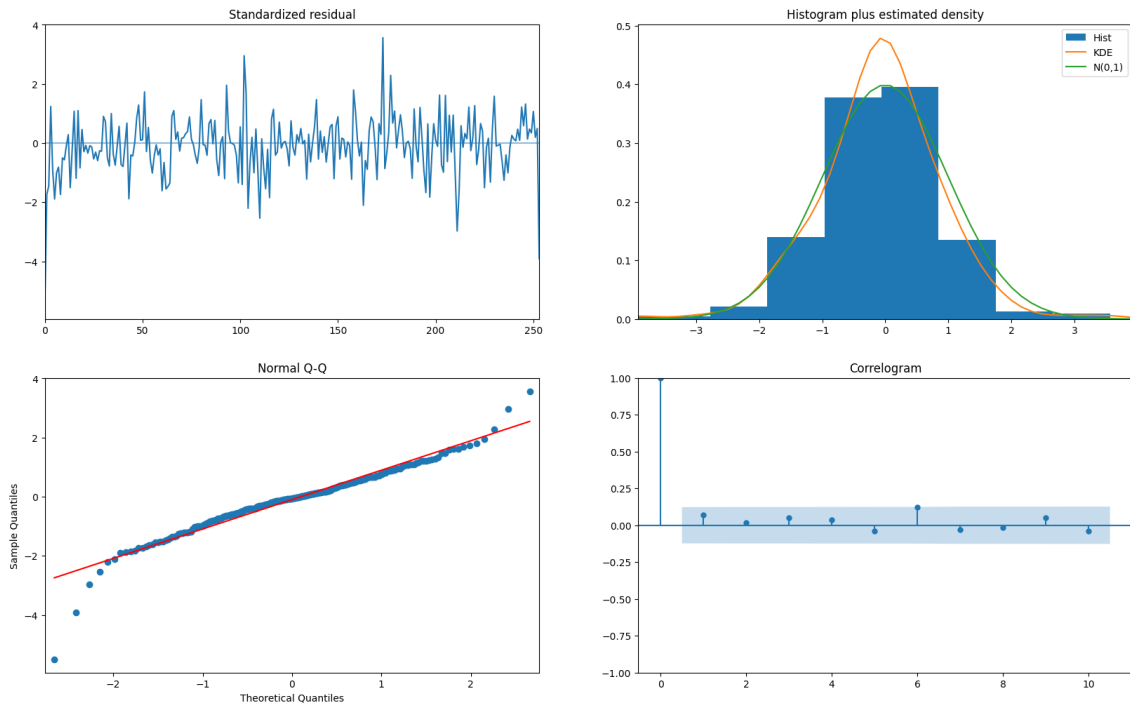


Figura 20: Diagnostica e analisi dei residui

Osservando i grafici della Figura 20, si può dire che:

### 1. Standardized Residuals

Questo grafico mostra gli errori del modello nel tempo, standardizzati in modo da avere una deviazione standard di 1.

- I residui fluttuano in modo casuale attorno alla linea dello zero. Non c'è alcun trend, ciclo o pattern evidente. La varianza appare costante.
- Concludendo, questo è l'aspetto ideale del “rumore bianco”, quindi il modello **ha estratto con successo tutta l'informazione prevedibile** dalla serie storica, lasciando solo un errore casuale e imprevedibile.

### 2. Histogram plus estimated density

Questo grafico confronta la distribuzione dei residui (istogramma blu e curva KDE arancione) con una distribuzione normale perfetta (curva verde  $N(0,1)$ ).

- La curva arancione (KDE) è centrata sullo zero e ha una forma a campana, ma è leggermente più “appuntita” e alta rispetto alla curva normale verde.
- Visivamente viene confermato il risultato del test di Jarque-Bera. I residui **non sono perfettamente normali**, ma sono molto vicini. La

forma più appuntita (leptocurtica) è comune nei dati reali e non invalida il modello per scopi di previsione.

### 3. Normal Q-Q Plot

Questo grafico confronta i quantili dei residui (punti blu) con i quantili di una distribuzione normale teorica (linea rossa).

- La stragrande maggioranza dei punti si allinea quasi perfettamente lungo la linea rossa. Si notano solo leggere deviazioni alle estremità, che corrispondono agli outlier.
- Questo è un risultato eccellente e rafforza la conclusione precedente. Nonostante la non-normalità statistica, la distribuzione **si avvicina molto a una distribuzione normale** per la maggior parte dei valori.

### 4. Correlogram (ACF dei Residui)

Questo grafico mostra se gli errori del modello sono correlati tra loro a diversi lag.

- **Nessuna** delle barre di autocorrelazione (a eccezione del lag 0, che è sempre 1) **supera in modo significativo l'area di confidenza blu**.
- Questo conferma visivamente il test di Ljung-Box ( $\text{Prob}(Q) > 0.05$ ): quindi **non c'è alcuna autocorrelazione residua significativa**. Il modello ha catturato tutta la dipendenza temporale (sia a breve termine sia stagionale) presente nei dati.

## 4.4 Addestramento del modello

Si addestrerà il modello dividendo il dataset in 80% training e 20% testing, così da poter simulare un addestramento sugli anni di misurazione (7) e valutare i risultati su un intero anno.

```
1 # Divisione in training e test set (80% training, 20% test)
2 train_size = int(len(crime_time_series) * 0.8)
3 train, test = crime_time_series[:train_size], crime_time_series[
    train_size:]
4
5 # Addestramento del modello SARIMA con i parametri trovati
6 model = SARIMAX(train['Total Incidents'],
7                 order=(1, 1, 1),
8                 seasonal_order=(2, 1, 0, 52),
9                 trend='n')
```

```

10 model_fit = model.fit(dispatch=False)
11
12 # Previsioni sul test set
13 forecast_object = model_fit.get_forecast(steps=len(test))
14 forecast_mean = forecast_object.predicted_mean
15 forecast_ci = forecast_object.conf_int()
16
17 # Calcolo degli errori
18 mse = mean_squared_error(test['Total Incidents'], forecast_mean)
19 mae = mean_absolute_error(test['Total Incidents'], forecast_mean)
20 rmse = np.sqrt(mse)

```

```

1 Mean Absolute Error (MAE): 76.28
2 Root Mean Squared Error (RMSE): 97.63

```

Listing 12: Valori degli Errori

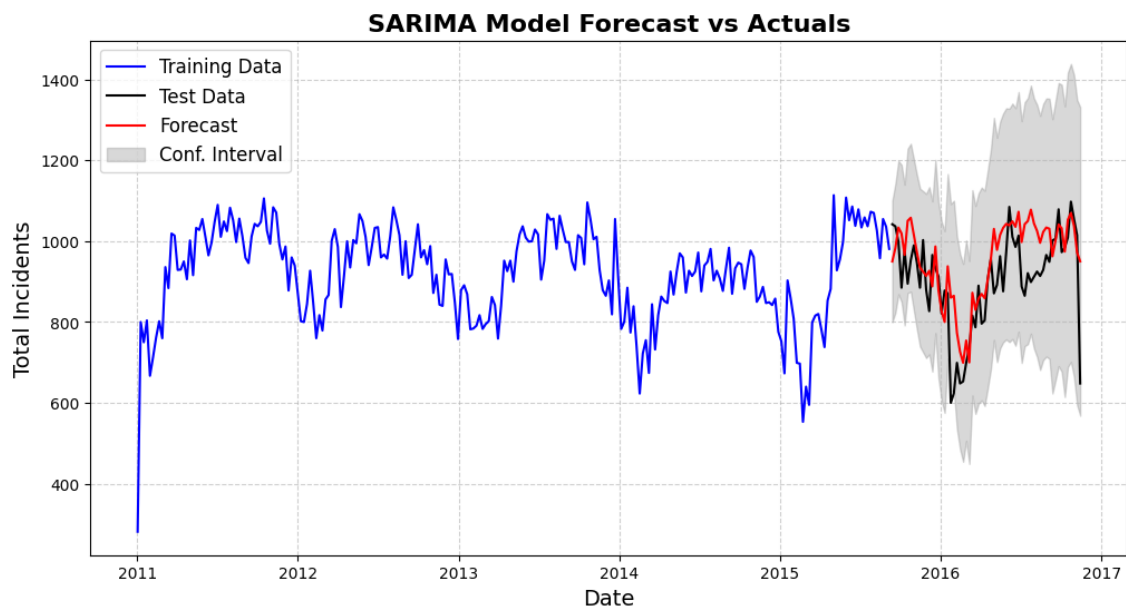


Figura 21: Modello SARIMAX

Analizzando il grafico del modello SARIMAX (Figura 21), si possono effettuare le seguenti considerazioni:

- **Previsione (Linea rossa):** la linea **cattura in modo eccellente la stagionalità** dei dati. Non è una linea piatta, ma segue correttamente i picchi e le valli del ciclo annuale. Ad esempio, riproduce bene il calo all'inizio del 2016 e la successiva risalita. Questo è il segnale più importante che il modello ha capito la struttura dei dati.

- **Dati Reali (Linea nera):** la linea dei dati di test rimane **quasi sempre all'interno dell'intervallo di confidenza**. Questo è esattamente ciò che si desidera: significa che il modello non solo fa una previsione puntuale, ma quantifica anche correttamente la propria incertezza.
- **Intervallo di confidenza (Area grigia):** l'intervallo di confidenza si **allarga leggermente man mano che la previsione si estende nel futuro**. Questo comportamento è corretto e desiderabile, poiché riflette il fatto che l'incertezza aumenta naturalmente su orizzonti temporali più lunghi.

Le metriche d'errore (Listato 12), inoltre, confermano l'impressione del grafico.

- **MAE (Mean Absolute Error): 76.28.** Significa che, in media, la previsione del modello si discosta dal valore reale di circa **76 incidenti a settimana**. Considerando che il numero di incidenti oscilla tra 800 e 1100 nel periodo di test, un errore medio di 76 è assolutamente ragionevole e indica una buona accuratezza.
- **RMSE (Root Mean Squared Error): 97.63.** L'RMSE è leggermente superiore al MAE. Questo è normale, poiché l'RMSE penalizza di più gli errori più grandi. Il fatto che i due valori siano relativamente vicini suggerisce che il modello non commette errori occasionali enormi, ma ha una performance consistente.

## 4.5 Miglioramento del modello SARIMAX

L'ultimo passaggio dell'analisi è quello di introdurre delle **variabili esogene** nel modello SARIMAX, per migliorare la precisione e la robustezza del modello. L'idea è di fornire non solo l'andamento passato della serie storica, ma anche altre informazioni contestuali che possono aiutarlo a spiegare le fluttuazioni e, di conseguenza, a fare previsioni più accurate.

La scelta delle variabili esogene è distinta tra due categorie differenti:

- **Variabili derivate dal dataset:**
  - *firearm\_rate*: percentuale di crimini con arma da fuoco ogni settimana.  
**Ragione:** i crimini con arma da fuoco possono avere un andamento stagionale o reagire a eventi particolari, come festività o tensioni sociali.
  - *inside\_rate*: percentuale di crimini all'interno di edifici ogni settimana.  
**Ragione:** differenziare crimini indoor da outdoor può catturare pattern

diversi; crimini indoor tendono ad avere correlazioni con stagionalità o condizioni climatiche, mentre quelli outdoor possono essere influenzati da fattori esterni come meteo o eventi pubblici.

- *district\_\**: distretti con maggior numero di incidenti ogni settimana. **Ragione:** alcune zone hanno tassi di criminalità più elevati; codificare i distretti come variabili dummy permette al modello di catturare effetti locali sulla serie temporale settimanale.

- **Variabili ingegnerizzate dalla serie storica:**

- *lag1*: numero di crimini nella settimana precedente. **Ragione:** i crimini spesso mostrano autocorrelazione settimanale; includere il lag aiuta il modello a prevedere l'andamento della settimana successiva basandosi sulla continuità dei dati.
- *rolling\_mean\_4w*: media mobile di 4 settimane. **Ragione:** cattura trend a breve periodo e attenua le fluttuazioni settimanali eccessive.
- *rolling\_std\_4w*: deviazione standard di 4 settimane. **Ragione:** misura la volatilità dei crimini; settimana con alta variabilità possono indicare eventi anomali.
- *month*: mesi dell'anno. **Ragione:** cattura la stagionalità annuale dei crimini (alcuni mesi possono avere più crimini per motivi climatici, festività o eventi locali).
- *is\_summer*: indicatore se la settimana appartiene ai mesi estivi (giugno-agosto). **Ragione:** durante l'estate possono cambiare i pattern di criminalità (più attività all'aperto, più eventi pubblici, vacanze scolastiche), quindi questa variabile aiuta a modellare effetti stagionali specifici.

```
1 # Copia della serie temporale originale
2 df = crime_time_series.copy()
3
4 # Creazione delle variabili esogene settimanali
5 crime_weekly = (
6     crime_data
7     .set_index('DateTime')
8     .resample('W')
9     .agg({
10         'Total Incidents': 'sum',
```



```

11     'Weapon': lambda x: (x == 'Firearm').mean(), # Percentuale di
        crimini con arma da fuoco
12     'Inside/Outside': lambda x: (x == 'Inside').mean(), #
        Percentuale di crimini all'interno
13     'District': lambda x: x.mode()[0] if len(x.mode()) else '
        Unknown' # Distretto pi frequente
14 })
15 .reset_index()
16 .rename(columns={
17     'Weapon': 'firearm_rate',
18     'Inside/Outside': 'inside_rate',
19     'District': 'top_district'
20 })
21 )
22
23 # One-hot encoding per la colonna 'top_district'
24 district_dummies = pd.get_dummies(crime_weekly['top_district'], prefix
    ='district', drop_first=True)
25 crime_weekly = pd.concat([crime_weekly, district_dummies], axis=1)
26 crime_weekly = crime_weekly.drop(columns=['top_district'])
27
28 # Creazione di ulteriori variabili esogene ingegnerizzate
29 crime_weekly['lag1'] = crime_weekly['Total Incidents'].shift(1)
30 crime_weekly['rolling_mean_4w'] = crime_weekly['Total Incidents'].
    rolling(4).mean()
31 crime_weekly['rolling_std_4w'] = crime_weekly['Total Incidents'].
    rolling(4).std()
32 crime_weekly['month'] = crime_weekly['DateTime'].dt.month
33 crime_weekly['is_summer'] = crime_weekly['month'].isin([6, 7, 8]).
    astype(int)
34
35 # Backfill dei valori mancanti per l'allineamento della serie
    temporale
36 cols_with_nans = ['lag1', 'rolling_mean_4w', 'rolling_std_4w']
37 for col in cols_with_nans:
38     crime_weekly[col] = crime_weekly[col].bfill()
39
40 # Rimozione delle righe con valori NaN generati dalle operazioni di
    shift e rolling
41 crime_weekly = crime_weekly.reset_index(drop=True)

```

```

42
43 # Unione delle variabili esogene con la serie temporale originale
44 df_exog = crime_weekly.drop(columns=['Total Incidents'])
45 df_final = df_exog.merge(df[['DateTime', 'Total Incidents']], on='
    DateTime', how='inner')
46
47 # Impostazione di 'DateTime' come indice
48 df_final = df_final.set_index('DateTime')
49
50 print("\n=== Dataset con variabili esogene ===")
51 print(df_final.head(5))
52 print("\n=== Colonne disponibili per SARIMAX ===")
53 print(list(df_final.columns))
54
55 # y = target, X = variabili esogene
56 y = df_final['Total Incidents']
57 X = df_final.drop(columns=['Total Incidents']).astype(float)

```

```

1  === Dataset con variabili esogene ===
2      firearm_rate inside_rate district_NORTHEASTERN \
3  DateTime
4  2011-01-02  0.0  0.0  False
5  2011-01-09  0.0  0.0  True
6  2011-01-16  0.0  0.0  True
7  2011-01-23  0.0  0.0  True
8  2011-01-30  0.0  0.0  True
9
10     district_NORTHERN district_SOUTHEASTERN district_SOUTHERN \
11  DateTime
12  2011-01-02  False  False  False
13  2011-01-09  False  False  False
14  2011-01-16  False  False  False
15  2011-01-23  False  False  False
16  2011-01-30  False  False  False
17
18     district_WESTERN lag1 rolling_mean_4w rolling_std_4w month \
19  DateTime
20  2011-01-02  False  281.0  658.75  253.028819  1
21  2011-01-09  False  281.0  658.75  253.028819  1
22  2011-01-16  False  800.0  658.75  253.028819  1
23  2011-01-23  False  750.0  658.75  253.028819  1
24  2011-01-30  False  804.0  755.25  63.756699  1
25

```

```

26         is_summer Total Incidents
27 DateTime
28 2011-01-02 0 281
29 2011-01-09 0 800
30 2011-01-16 0 750
31 2011-01-23 0 804
32 2011-01-30 0 667
33
34 === Colonne disponibili per SARIMAX ===
35 ['firearm_rate', 'inside_rate', 'district_NORTHEASTERN', 'district_NORTHERN',
    ↪ 'district_SOUTHEASTERN', 'district_SOUTHERN', 'district_WESTERN', '
    ↪ lag1', 'rolling_mean_4w', 'rolling_std_4w', 'month', 'is_summer', '
    ↪ Total Incidents']

```

Listing 13: Risultati calcolo variabili esogene

Le metriche di errore dimostrano un **miglioramento netto e significativo** rispetto al modello SARIMAX originale.

| Metrica | SARIMAX (Prec.) | SARIMAX (Nuovo) | Miglioramento |
|---------|-----------------|-----------------|---------------|
| MAE     | 76.28           | <b>51.56</b>    | <b>-32.4%</b> |
| RMSE    | 97.62           | <b>67.30</b>    | <b>-31.1%</b> |

- **Mean Absolute Error (MAE): 51.56.** Questo risultato è molto buono. Significa che, in media, le previsioni del nuovo modello si discostano dal valore reale di soli **51 incidenti**, contro i 76 del modello precedente. L'errore medio è stato ridotto di oltre il **32%**.
- **Root Mean Squared Error (RMSE): 67.30.** Anche l'RMSE, che penalizza maggiormente gli errori più grandi, mostra una riduzione drastica di oltre il **31%**. Questo indica che il modello non solo è mediamente più accurato, ma compie anche meno errori di grande entità.

Questi numeri confermano che l'aggiunta di variabili esogene contestuali ha aumentato drasticamente il potere predittivo del modello.

Il confronto visivo tra i dati reali e le previsioni del modello SARIMAX rafforza ulteriormente le conclusioni tratte dalle metriche (Figura 22).

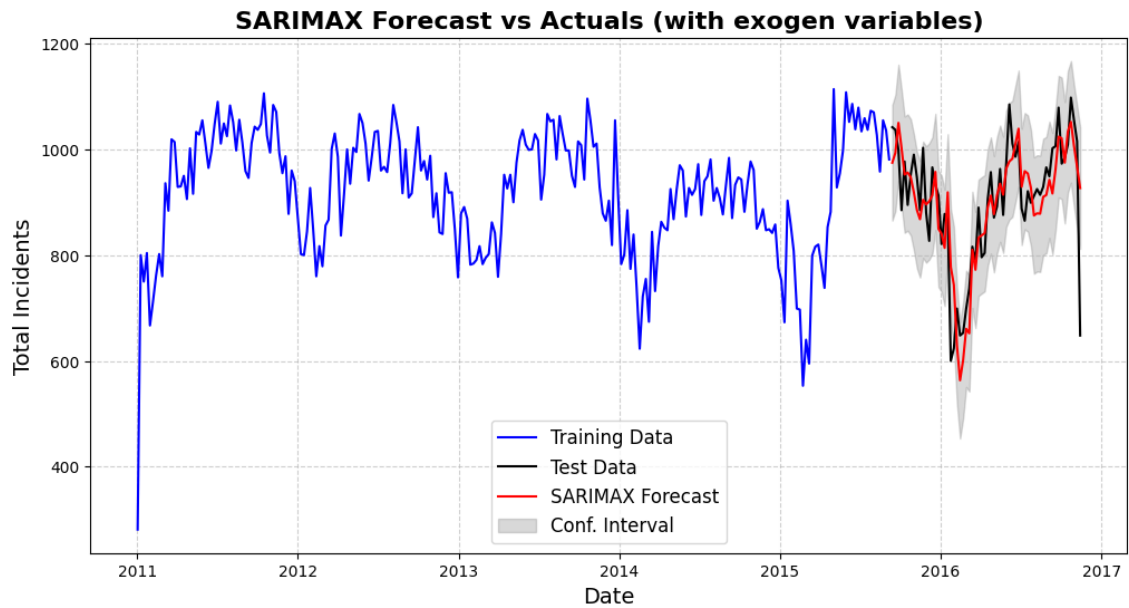


Figura 22: Modello SARIMAX migliorato

Dal grafico si evince che:

1. **Aderenza ai Dati Reali:** La linea rossa della previsione (SARIMAX Forecast) non si limita più a seguire il trend stagionale generale. Ora è in grado di **capturare i picchi e le valli settimanali** con notevole precisione, seguendo da vicino la linea nera dei dati di test.
2. **Spiegazione della Varianza:** Il modello sta chiaramente utilizzando le informazioni fornite dalle variabili esogene (come la media mobile e la stagionalità estiva) per **spiegare le fluttuazioni a breve termine**, un compito che il modello SARIMA originale non poteva assolvere.
3. **Affidabilità della Previsione:** L'intervallo di confidenza (area grigia) rimane ragionevolmente stretto, indicando una buona sicurezza del modello nelle sue previsioni.

Plottando i residui, è possibile inoltre analizzarli per avere delle osservazioni aggiuntive.

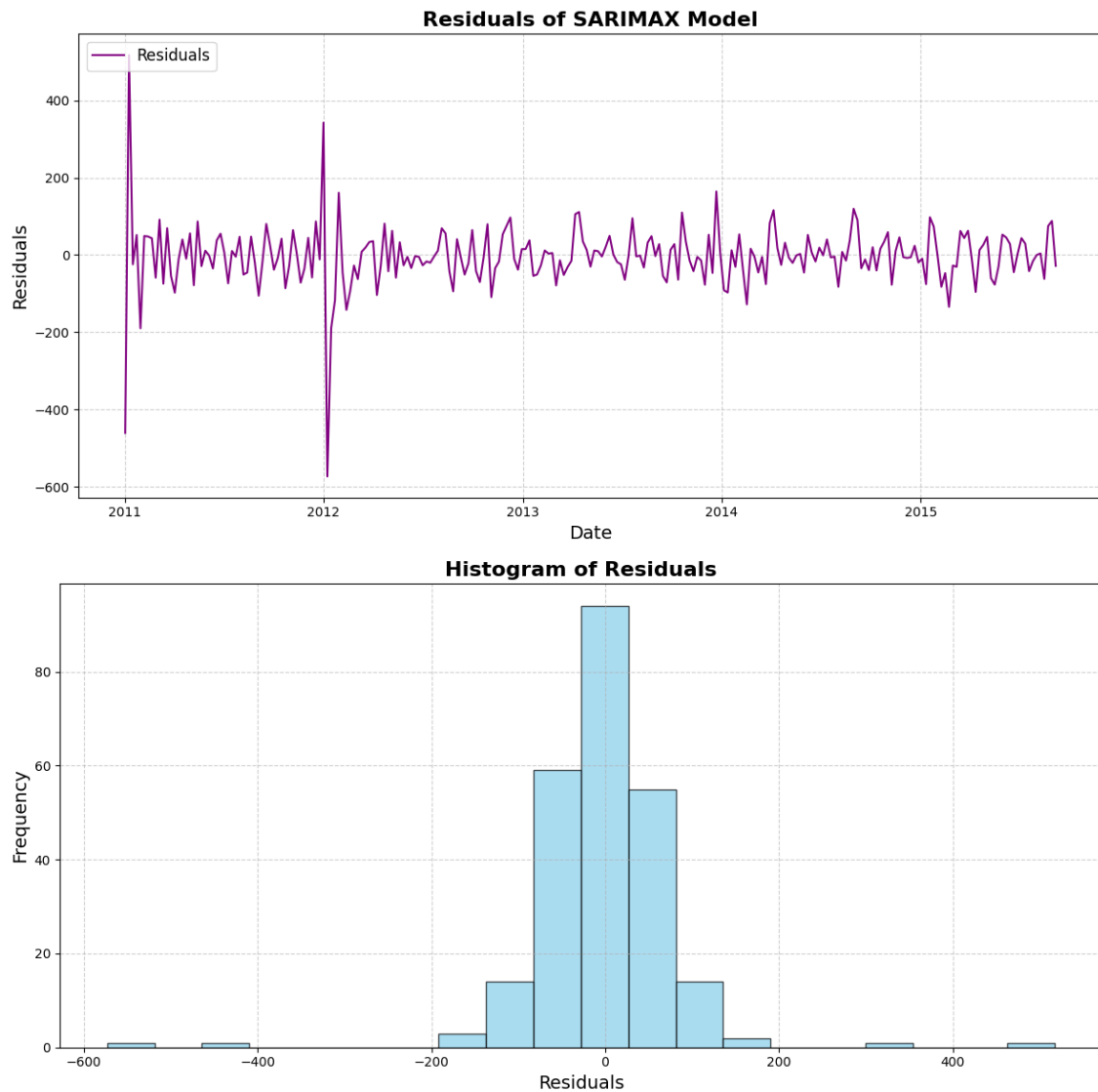


Figura 23: Grafici dei residui

Osservando i due grafici (Figura 23), si può notare che:

### 1. Grafico dei residui

Dall'analisi del grafico emergono tre osservazioni positive:

- **Media Attorno a Zero:** I residui oscillano costantemente attorno al valore zero, indicando che il modello non soffre di un *bias* sistematico (non tende né a sovrastimare né a sottostimare).
- **Varianza Costante (Omoschedasticità):** L'ampiezza delle oscillazioni è ragionevolmente stabile per tutto il periodo, suggerendo che la varianza degli errori sia costante.

- **Assenza di Pattern:** Non sono visibili cicli o schemi prevedibili. Gli errori appaiono casuali, confermando che il modello ha catturato con successo la struttura temporale e l'autocorrelazione presenti nei dati.

## 2. Istogramma dei residui

L'istogramma conferma ulteriormente la bontà del modello:

- **Forma a Campana:** La distribuzione dei residui ha una chiara e simmetrica forma a campana, molto simile a una distribuzione normale.
- **Centratura sullo Zero:** Il picco della distribuzione è centrato esattamente sullo zero, rafforzando l'idea che l'errore medio del modello sia nullo.

## 4.6 Conclusioni ed osservazioni

Questo progetto ha mostrato come l'analisi delle serie temporali dei crimini a Baltimora possa essere notevolmente migliorata introducendo variabili esogene nel modello SARIMAX. Partendo da un SARIMA di base, che catturava solo l'andamento passato, abbiamo osservato che previsioni apparentemente perfette potevano nascondere data leakage, sottolineando l'importanza di selezionare feature rilevanti.

Grazie a un'attenta ingegnerizzazione delle variabili, come il tasso di crimini con arma da fuoco, indicatori stagionali e statistiche delle settimane precedenti, il modello ha ridotto il MAE di oltre il 32%, con residui che si comportano come rumore bianco, confermando la validità statistica.

In sintesi, la precisione delle previsioni dipende più dalla **qualità delle informazioni fornite** che dalla complessità del modello: integrare contesto e pattern recenti ha reso il SARIMAX uno strumento predittivo più accurato e affidabile.